Diese Arbeit wurde vorgelegt am Lehrstuhl für Chemische Verfahrenstechnik
The present work was submitted to Chair of Chemical Process Engineering

Dichte 3D-Echtzeit-Rekonstruktion für die endoskopische Chirurgie mittels monokularer Sequenzen durch ein Transformer-basiertes Feed-Forward neuronales Netz

Real-Time Dense 3D Reconstruction for Endoscopic Surgery using Monocular Sequences via a Transformer-Based Feed-Forward Neural Network

Masterarbeit
Master Thesis

von / presented by
Guan, Zhouyi (406259)

Betreuer*in / Supervisor
Yoo, Sang-Whon, M. Sc.

Prof. Dr.-Ing. Matthias Wessling

Aachen, DATE

Abstract

Osteoarthritis remains a global socioeconomic burden characterized by the irreversible degradation of Articular Cartilage (AC). Within the framework of the DWI Leibniz-Institute for Interactive Materials, research is directed toward developing fully automatic in vivo 3D bioprinting to overcome the limitations of traditional ex vivo scaffolding. Conventional dense 3D reconstruction techniques in endoscopic scenarios have struggled with weak structural textures, leading to insufficient robustness and precision. Furthermore, existing algorithmic frameworks have remained computationally heavy with prohibitive processing times, and their performance has been frequently compromised by dynamic surgical instruments or flowing tissues that obstruct the surgical field. This study aims to design a high-precision, real-time dense SLAM framework capable of maintaining robust 3D reconstruction even in the presence of intraoperative occlusions and dynamic disturbances. A Transformer-based feed-forward neural network was developed to perform high-fidelity dense 3D reconstruction. Besides a dedicated motion head was integrated and trained to identify and decouple dynamic surgical instruments and flowing tissues from the scenes. Furthermore, a real-time SLAM framework was constructed utilizing $SL(4)$ manifold representation. /TODO: Add quantitative results/

Assignment

Artificial Intelligence (AI) Statement

This thesis has been partially rewritten using an artificial intelligence (AI) model to improve readability. All ideas, considerations, and content are original and created by the author. Therefore, AI systems are not concerned with intellectual property. The AI usage rules stipulated by the Faculty of Mechanical Engineering of RWTH Aachen University on July 29, 2025, have been fully satisfied.

Contents

1 Introduction **1**

2 Theoretical Background **3**
2.1 Medical Background . . . 3
2.1.1 Articular Cartilage . . . 3
2.1.2 Osteoarthritis . . . 5
2.1.3 3D Bioprinting Techniques . . . 6
2.2 Geometric and Optimization Foundations . . . 8
2.2.1 Camera Model . . . 8
2.2.2 Structure from Motion (SfM) . . . 9
2.2.3 Multi-View Stereo (MVS) . . . 10
2.2.4 Point Cloud Registration . . . 10
2.2.5 Robust Estimation with RANSAC . . . 15
2.2.6 Global Optimization . . . 15
2.2.7 Depth-ray Representation . . . 18
2.2.8 SLAM . . . 19
2.3 Learning-Based Dense 3D Reconstruction . . . 21
2.3.1 Vision Transformer (ViT) . . . 22
2.3.2 Self-Distillation with No Labels (DINO) . . . 22
2.3.3 Dense Prediction Transformer (DPT) . . . 23
2.3.4 Feed-Forward Transformer-Based 3D Reconstruction . . . 24
2.4 State of the Art . . . 31

3 Materials and Methods **34**
3.1 DynaDA3 . . . 34
3.1.1 Preprocessing . . . 34
3.1.2 Architecture . . . 36
3.1.3 Training . . . 40
3.2 DynaDA3-SLAM . . . 42
3.2.1 System Principle and Core Components . . . 42
3.2.2 Image Scheduling and Keyframe Selection . . . 43
3.2.3 Learned Front-End Inference and Dense Submap Reconstruction . . . 43
3.2.4 Inter-Submap Registration and Loop-Closure Constraints . . . 44
3.2.5 Global Graph Optimization and Map Update . . . 45

4 Results and Discussion **46**
4.1 Pose Estimation Results . . . 46

4.2 Depth Reconstruction Results 47
4.3 3D Scene Quality Results 48
4.4 SLAM Results 48
4.4.1 Sparse SLAM Results on Joint Sequences 49
4.4.2 Dense SLAM Results 49
4.5 Dynamic Obstacle Prediction Results 50
4.6 Discussion 50

5 Conclusion and Outlook **52**

Bibliography **53**

List of Figures **57**

List of Tables **58**

A Appendix **59**

1 Introduction

Osteoarthritis (OA) stands as a premier cause of global disability, characterized by the irreversible degradation of Articular Cartilage (AC). Due to the avascular nature of AC, its intrinsic repair capacity is remarkably limited; even minor focal lesions often fail to heal, eventually leading to chronic pain and impaired mobility. While traditional ex vivo scaffolding techniques have been widely researched, recent advancements in medical robotics and materials science are paving the way for direct, in vivo 3D bioprinting. As envisioned within the major initiative at the DWI Leibniz-Institute for Interactive Materials, a fully automatic robotic process for in situ cartilage repair offers a transformative alternative by minimizing contamination risks and bypassing the time-consuming nature of conventional methods.[1]

The success of in vivo bioprinting is fundamentally predicated on the precise 3D scanning and geometry estimation of the target lesion. Traditionally, pre-operative Magnetic Resonance Imaging (MRI) has been the gold standard; however, it suffers from systematic underestimation of lesion thickness and limited out-of-plane resolution. While photogrammetry presents a promising non-contact alternative, conventional dense 3D reconstruction algorithms often falter in the challenging endoscopic environment. These techniques are frequently hampered by weak textural features, prohibitive computational overhead, and the presence of dynamic occlusions—such as surgical instruments and flowing tissues—which compromise both the accuracy and the robustness required for real-time clinical intervention.[1]

Building upon the foundational work at DWI, this thesis aims to bridge the gap between static photogrammetry and real-time surgical navigation. The primary objective is to develop a high-precision, real-time dense 3D reconstruction and SLAM framework. By leveraging Transformer-based architectures and advanced manifold optimization, this work seeks to provide a robust spatial mapping solution that can effectively handle intraoperative occlusions and provide the necessary geometric intelligence for autonomous robotic bioprinting.

The proposed pipeline introduces a Transformer-based feed-forward neural network designed for

high-fidelity geometry estimation, capturing global dependencies that traditional methods often miss. To ensure clinical reliability, a dedicated motion masking head is integrated to identify and decouple dynamic surgical disturbances from the static cartilage surface. Furthermore, the framework achieves high-efficiency real-time performance by utilizing an $SL(4)$ (Special Linear Group) manifold representation for SLAM, allowing for robust tracking under complex projective transformations inherent in monocular endoscopy.

The remainder of this thesis is structured as follows:

Section **??** discusses the medical background of knee AC and OA, emphasizing state-of-the-art treatment and 3D bioprinting techniques.

Section 2.2 introduces the geometric foundations of 3D vision, common visual tasks, and the corresponding estimation and optimization algorithms.

Section 2.3 introduces learning-based dense 3D reconstruction technologies.

Section 3.1 details the architectural design and training process of the proposed Transformer-based feed-forward neural network.

Section 3.2 outlines the construction of the real-time dense SLAM framework built upon the aforementioned model.

Chapter 4 analyzes the experimental results obtained from the pipeline, using well plates and simulated environments as precursors.

Chapter 5 concludes the thesis with a summary of findings and an outlook on potential optimizations for clinical application.

2 Theoretical Background

2.1 Medical Background

This section introduces the medical background of the knee joint that is relevant to this work. First, Articular Cartilage (AC) and its structural properties are described, followed by Osteoarthritis (OA) as a degenerative joint disease. Based on this medical context, the TriggerINK project is outlined as a robotic 3D bioprinting approach for cartilage repair, which motivates the subsequent technical requirements of the present work.[1, 2]

2.1.1 Articular Cartilage

The human body is composed of various cells that not only form organs but also define different types of organic tissue. The four fundamental tissue types are nervous, muscular, epithelial, and connective tissue. Among these, connective tissue is the most widespread and diverse, encompassing fat tissue, blood, fibrous tissue, bone marrow, and cartilage. These connective tissues are highly specialized and rich in matrix, which ensures cohesion, mechanical support, and protection of the body's organs and structures.

Among them, cartilage is noted for its combination of flexibility and strength, and it is present in multiple locations throughout the body. Its dense matrix, characterized by a gelatinous base, is heavily infused with collagens and elastic fibers, allowing its categorization into several types based on structural differences:

- **Hyaline cartilage** displays a bluish-white hue and is rich in collagen fibrils and Glycosaminoglycans (GAG), predominantly associated with the skeletal system.

- **Elastic cartilage** is identified by its yellowish color and is found in the external ear and

epiglottis, characterized by a matrix composed of extensive networks of elastic fibers.

- **Fibrous cartilage** is the strongest cartilage type and consists of a sponge-like matrix densely filled with collagen. Key components such as menisci, calli, and intervertebral discs are made up of this cartilage variant.

Hyaline cartilage is the most commonly found type in the body, with a specialized form known as Articular Cartilage (AC) located in synovial joints.[3] It plays a crucial role in the musculoskeletal system, particularly in the knee. AC has an average thickness of 2–4 mm and is composed of a dense Extracellular Matrix (ECM) with a sparse distribution of highly specialized cells called chondrocytes. The anatomical relationship between the major structures of the knee joint and the articular cartilage is illustrated in Figure 2.1. In particular, the figure highlights the articulation region where the cartilage layer covers the joint surfaces and enables smooth motion under load.

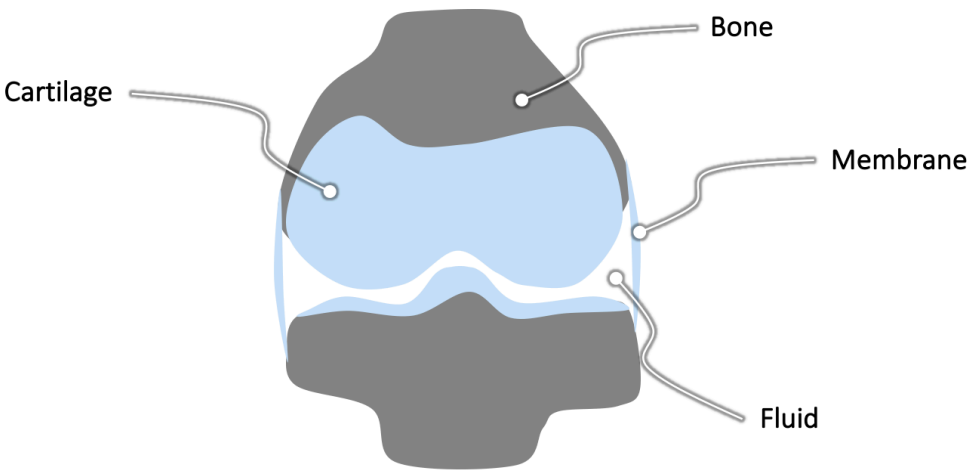

Figure 2.1: Schematic overview of the knee joint and the anatomical location of articular cartilage.

The ECM is mainly composed of water, collagen, and proteoglycans, where proteoglycans are formed by GAG chains covalently bound to a protein core. Collagens are primarily responsible for the durability of AC, while proteoglycans play a critical role in resilience to compressive loads and elasticity.[4, 5] Together, these components result in the rubber-like mechanical functionality of AC, providing a smooth, lubricated joint surface and facilitating load distribution. This significantly reduces contact stress during movement and protects the underlying bone from permanent damage.

The structure of AC in synovial joints is divided into four zones: superficial, middle, deep, and calcified cartilage. Each zone exhibits distinct structural properties and fulfills specific biomechanical functions.[1, 3]

Cartilage is avascular, meaning that there is no direct blood supply and chondrocytes rely on nutrient diffusion from the surrounding environment. The compressive forces that regularly act on cartilage further promote nutrient transport. This indirect supply mechanism is a major factor behind the slow turnover of the extracellular matrix and the limited repair capacity of cartilage.[2]

2.1.2 Osteoarthritis

Traditionally, Osteoarthritis (OA) was viewed as a disease primarily affecting AC, but it is now recognized as involving the entire joint, with interconnected components all susceptible to OA-related degenerative processes. This broader understanding emphasizes the complex nature of OA, which is influenced by a range of risk factors including age, mechanical stimuli, natural wear and tear, obesity, occupational factors, and lifestyle choices. Globally, around 240 million people suffer from symptomatic OA that limits daily activity, predominantly in the knee joint followed by the hip joint. For instance, nearly 30% of individuals over 45 years old show knee OA on X-rays, and about half of these cases are symptomatic. This makes OA the most prevalent and most clinically relevant type of arthritis worldwide.[2]

In general, OA starts with molecular disturbances in joint tissue metabolism and progresses to anatomical and physiological changes. These changes include focal lesions in the cartilage, subchondral bone remodeling, osteophyte formation, joint space narrowing, and synovial inflammation, which in turn can lead to synovial hyperplasia and significant health problems. Within the scope of this work, only focal lesions are discussed in further detail.[2] Figure 2.2 shows a representative cartilage defect in the knee joint. This localized lesion is the type of pathology addressed in the present repair scenario and therefore constitutes the primary target structure for the subsequent reconstruction and treatment workflow.

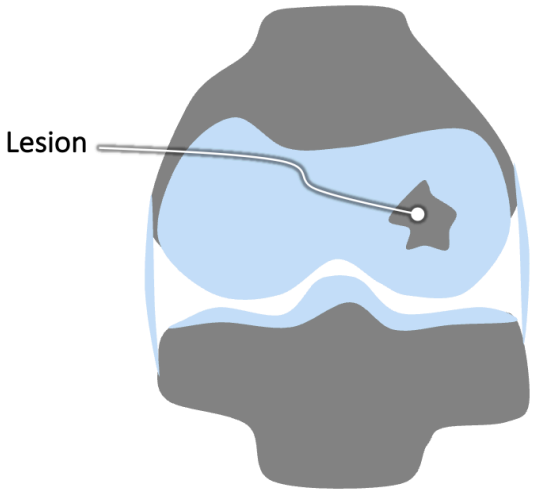

Figure 2.2: Illustration of a focal cartilage lesion in the knee joint.

2.1.3 3D Bioprinting Techniques

One state-of-the-art strategy for cartilage repair relies on a cell-based and partially ex vivo workflow. In such an approach, healthy cartilage cells are first harvested in an initial surgical step, then cultivated outside the body in the laboratory, and after a prolonged preparation period they are seeded onto a supporting matrix before being implanted into the cartilage defect during a second surgery. While this strategy can provide biologically relevant repair material, it also comes with clear disadvantages, including the need for two separate operations, extended treatment time, increased logistical complexity, and the risk of contamination during ex vivo processing.[1]

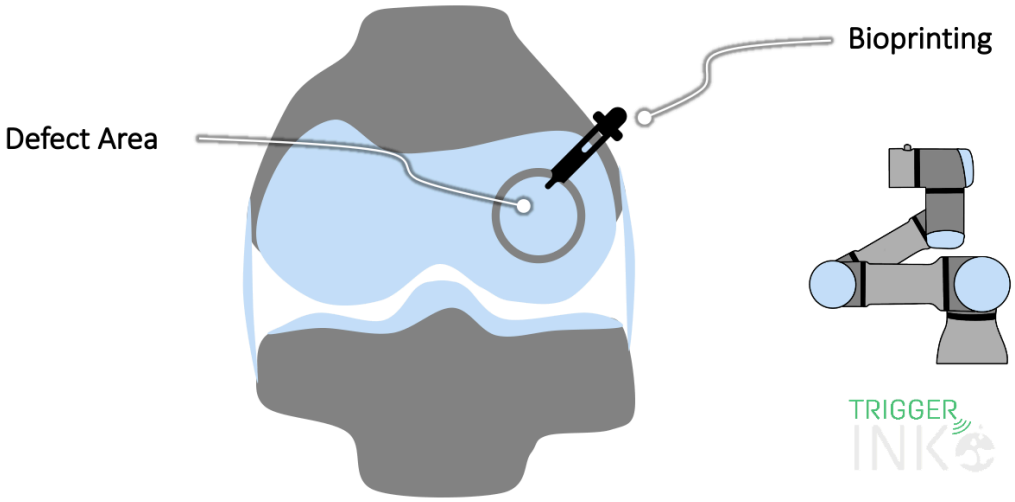

Figure 2.3: Comparison between a conventional multi-step cartilage repair workflow and the TriggerINK single-surgery robotic bioprinting concept.

In contrast, the TriggerINK project pursues an in vivo robotic bioprinting strategy that aims to perform the repair within a single surgery using a fully functional synthetic cartilage tissue as bioink.[1] As summarized in Figure 2.3, the vision is to replace the conventional two-stage workflow by a fully automatic process in which defect assessment, geometry acquisition, and material deposition are integrated into one robotic procedure. Within this overall concept, the present thesis addresses the visual information processing that must be solved before automated printing becomes feasible.

In principle, such a repair procedure can be realized either through an open surgical approach or through a minimally invasive approach. Compared with open surgery, the minimally invasive setting offers several important advantages, including reduced recovery time, minimal trauma, and less postoperative pain. For this reason, the following technical discussion focuses on the minimally invasive scenario.

However, the minimally invasive setting also introduces major challenges. Since the intervention takes place inside the joint cavity, the defect cannot be observed directly and its geometry cannot be scanned directly from outside the body. Instead, the available intraoperative information is limited to the video stream delivered by the endoscope.

Consequently, the processing pipeline must not only reconstruct the defect region with high geometric accuracy, but also support real-time localization and navigation of the endoscope

and surgical tools inside the surgical scene. This leads directly to the technical goals of the present thesis: high-precision dense 3D reconstruction together with real-time localization and navigation. A strongly related technical foundation for these requirements is Simultaneous Localization and Mapping (SLAM), which is introduced in a later section.

2.2 Geometric and Optimization Foundations

This section introduces the foundations of 3D geometry together with several common visual tasks in computer vision. It first presents the basic camera and multi-view geometric models, then discusses representative tasks such as Structure-from-Motion (SfM), Multi-View Stereo (MVS), and SLAM, and finally reviews the core estimation and optimization algorithms that are commonly used to solve them, including RANSAC, bundle adjustment, and pose transformations.

2.2.1 Camera Model

Rigid Body Transformation

Camera motion is modeled as a rigid body transformation. A point $\mathbf{P}_w \in \mathbb{R}^3$ in the world frame is transformed into the camera coordinate frame $\mathbf{P}_c$ via a rotation $\mathbf{R} \in SO(3)$ and a translation $\mathbf{t} \in \mathbb{R}^3$:

$$\mathbf{P}_c = \mathbf{R}\mathbf{P}_w + \mathbf{t}$$

The pose is represented using the Special Euclidean group $\mathbf{T}_{cw} \in SE(3)$:

$$\mathbf{T}_{cw} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}$$

Pinhole Camera Projection

We utilize the standard pinhole camera model. The projection function $\pi : \mathbb{R}^3 \to \mathbb{R}^2$ maps a 3D point $\mathbf{P}_c = [x_c, y_c, z_c]^\top$ to 2D pixel coordinates $\mathbf{u} = [u, v]^\top$:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K}\mathbf{P}_c = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix}$$

where s is the depth factor (z_c), f_x and f_y are the focal lengths, and (c_x, c_y) denotes the principal point.

Epipolar Geometry Constraint

For two views observing the same 3D point, the geometric relationship is encoded by the Essential Matrix $\mathbf{E}$. Let $\mathbf{x}_1$ and $\mathbf{x}_2$ be the normalized coordinates of matched feature points in two frames. They satisfy

$$\mathbf{x}_2^\top \mathbf{E} \mathbf{x}_1 = 0, \quad \text{where } \mathbf{E} = [\mathbf{t}]_\times \mathbf{R}$$

where $[\mathbf{t}]_\times$ is the skew-symmetric matrix of the translation vector $\mathbf{t}$. This constraint is commonly utilized during pose initialization.

2.2.2 Structure from Motion (SfM)

Structure from Motion is a classic computer vision problem [45, 77, 80] that involves estimating camera parameters and reconstructing sparse point clouds from a set of images of a static scene captured from different viewpoints. The traditional SfM pipeline [2, 36, 70, 94, 103, 134] consists of multiple stages, including image matching, triangulation, and bundle adjustment. COLMAP [94] is the most popular framework based on the traditional pipeline.[6]

Figure2.4 illustrates a fundamental SfM framework.The pipeline starts from multi-view image inputs, establishes inter-image correspondences through feature matching and geometric verification, and then incrementally estimates camera poses and 3D scene structure. The reconstruction is continuously refined through triangulation and bundle adjustment, resulting in a globally consistent 3D representation.

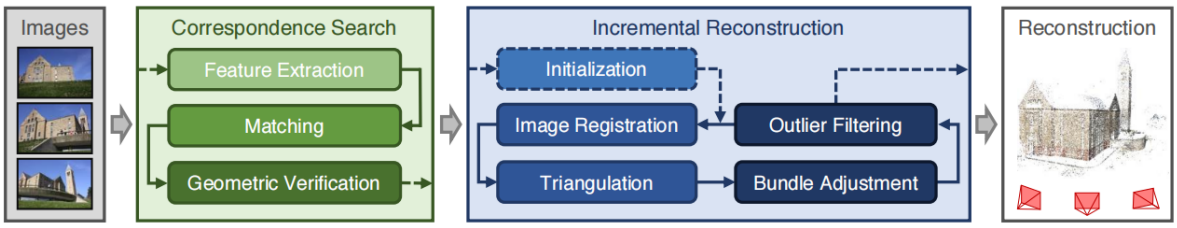

Figure 2.4: Structure-from-Motion pipeline

2.2.3 Multi-View Stereo (MVS)

Multi-view Stereo (MVS) aims to densely reconstruct the geometry of a scene from multiple overlapping images, typically assuming known camera parameters which are often estimated via SfM [6]. The fundamental principle underlying MVS is the Photo Consistency Assumption, which posits that a 3D point in space should exhibit a similar appearance (color or intensity) when projected onto different image planes, provided the surface is approximately Lambertian. To efficiently identify corresponding points across views, MVS leverages Epipolar Geometry. Instead of searching the entire image domain, pairwise matching is constrained to epipolar lines determined by the relative camera poses, thereby significantly reducing computational complexity. Based on these geometric constraints, the algorithm estimates a depth map for each reference image by aggregating information from neighboring views. Finally, in the Fusion stage, these individual depth maps are integrated into a globally consistent point cloud or mesh, filtering out outliers to produce a high-fidelity dense estimation of the scene geometry.

2.2.4 Point Cloud Registration

Point cloud registration seeks a transformation that aligns one set of 3D points with another. Depending on the geometric assumptions of the problem, different transformation groups are used. The most common case is a rigid transformation in $SE(3)$, which preserves distances and is appropriate when the two point sets describe the same object or scene up to rotation and translation. When the relative scale is unknown, as in monocular reconstruction or map alignment, a similarity transformation in $Sim(3)$ is required. More general formulations may further employ projective transformations represented by matrices in $SL(4)$, which can model perspective distortions in a homogeneous 3D representation and have recently appeared in learning-based geometric pipelines. In practice, these models form a hierarchy from physically

interpretable rigid alignment to more flexible but less constrained projective alignment.

$SE(3)$ Point Alignment: ICP with SVD Solution

For rigid registration, let the source point set be $P = \{p_i \in \mathbb{R}^3\}_{i=1}^N$ and the target point set be $Q = \{q_i \in \mathbb{R}^3\}_{i=1}^N$. Under known correspondences, the point-to-point alignment problem seeks the rigid transformation $(\mathbf{R}, \mathbf{t})$ that minimizes the squared Euclidean distance

$$\min_{\mathbf{R} \in SO(3),\, \mathbf{t} \in \mathbb{R}^3} \sum_{i=1}^{N} \|\mathbf{R}p_i + \mathbf{t} - q_i\|^2,$$

where $SO(3) = \{\mathbf{R} \mid \mathbf{R}^\top \mathbf{R} = \mathbf{I},\ \det(\mathbf{R}) = 1\}$. In the Iterative Closest Point (ICP) algorithm, these correspondences are updated iteratively by nearest-neighbor search, and the rigid alignment is recomputed after each update [7, 8].

For a fixed rotation $\mathbf{R}$, the optimal translation is obtained by aligning the centroids of the two point sets. Defining the centroids

$$\bar{p} = \frac{1}{N} \sum_{i=1}^{N} p_i, \qquad \bar{q} = \frac{1}{N} \sum_{i=1}^{N} q_i,$$

the translation can be written as

$$\mathbf{t}^*(\mathbf{R}) = \bar{q} - \mathbf{R}\bar{p}.$$

Thus, after centering the point sets with $p_i' = p_i - \bar{p}$ and $q_i' = q_i - \bar{q}$, the optimization reduces to

$$\min_{\mathbf{R} \in SO(3)} \sum_{i=1}^{N} \|\mathbf{R}p_i' - q_i'\|^2.$$

Expanding the squared norm shows that the terms independent of $\mathbf{R}$ can be discarded, yielding the equivalent trace maximization problem

$$\max_{\mathbf{R}\in SO(3)} \operatorname{tr}(\mathbf{R}\mathbf{H}), \qquad \mathbf{H} = \sum_{i=1}^{N} p_i'(q_i')^\top,$$

where $\mathbf{H}$ is the cross-covariance matrix between the centered point sets.

The optimal rotation is then obtained from the singular value decomposition

$$\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top.$$

To avoid reflections and enforce $\det(\mathbf{R}) = 1$, a correction matrix

$$\mathbf{S} = \operatorname{diag}(1, 1, \det(\mathbf{V}\mathbf{U}^\top))$$

is introduced, giving the closed-form solution

$$\mathbf{R}^* = \mathbf{V}\mathbf{S}\mathbf{U}^\top, \qquad \mathbf{t}^* = \bar{q} - \mathbf{R}^*\bar{p}.$$

This SVD-based solver provides the rigid update used inside each ICP iteration and is efficient enough to be recomputed repeatedly as correspondences change.

In one ICP iteration, the current estimate $(\mathbf{R}, \mathbf{t})$ first transforms the source point cloud, then nearest-neighbor search establishes tentative correspondences, and finally the SVD solution above yields an incremental rigid update. The global estimate is composed as

$$\mathbf{R} \leftarrow \mathbf{R}_\Delta \mathbf{R}, \qquad \mathbf{t} \leftarrow \mathbf{R}_\Delta \mathbf{t} + \mathbf{t}_\Delta.$$

The process repeats until the residual change falls below a threshold or a maximum number of iterations is reached.

$Sim(3)$ Alignment: Umeyama Algorithm

Rigid alignment is insufficient when the two point sets differ by an unknown global scale. This is common in monocular vision, where reconstruction is only determined up to scale. In that case, the alignment is modeled by a similarity transformation with scale factor $s > 0$,

$$\mathbf{x}' = s\mathbf{R}\mathbf{x} + \mathbf{t}, \qquad \mathbf{S} = \begin{bmatrix} s\mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \in Sim(3).$$

Given corresponding points $\{p_i\}$ and $\{q_i\}$, the least-squares objective becomes

$$\min_{s,\mathbf{R},\mathbf{t}} \sum_{i=1}^{N} \|p_i - (s\mathbf{R}q_i + \mathbf{t})\|^2.$$

The standard closed-form solution is the Umeyama algorithm [9]. As in the rigid case, the method first computes the centroids μ_p and μ_q, centers the point sets, and forms a covariance matrix from the centered coordinates. The rotation is again recovered by SVD, while the scale is estimated by comparing the aligned covariance with the variance of the source points. In compact form, the scale and translation can be written as

$$s^* = \frac{\sum_{i=1}^{N} (p_i')^\top \mathbf{R} q_i'}{\sum_{i=1}^{N} \|q_i'\|^2}, \qquad \mathbf{t}^* = \mu_p - s^* \mathbf{R} \mu_q.$$

Compared with the $SE(3)$ case, the key difference is that the recovered transformation not only aligns orientation and position but also compensates for accumulated scale drift. This makes $Sim(3)$ alignment particularly important for monocular SLAM and map merging.

$SL(4)$ and More General Projective Alignment

In projective geometry, $SL(4)$ denotes the special linear group of real 4×4 matrices with unit determinant. Unlike $SE(3)$ and $Sim(3)$, which model rigid and similarity transformations in Euclidean space, $SL(4)$ acts on homogeneous 3D coordinates and can represent a full projective correction with 15 degrees of freedom. This additional flexibility becomes relevant when a reconstruction is only projectively consistent, as may happen with uncalibrated monocular inputs or feed-forward reconstruction models [6, 10]. In these settings, the discrepancy between two local reconstructions may include not only rotation, translation, and scale, but also shear, anisotropic stretch, and perspective distortion that cannot be captured by $Sim(3)$ alone.

More generally, suppose two projectively related reconstructions provide corresponding homogeneous 3D points X_i and Y_i. The goal is to estimate a projective transformation $\mathbf{H}$ such that

$$X_i \sim \mathbf{H} Y_i, \qquad \mathbf{H} \in \mathbb{R}^{4\times 4},$$

where $\sim$ denotes equality up to scale in homogeneous coordinates. Since a 4×4 matrix has 16 entries but one global scale is irrelevant, the resulting projective alignment problem has 15 effective degrees of freedom.

The estimation can be formulated as a direct linear transform (DLT) problem. By vectorizing the unknown matrix into $h \in \mathbb{R}^{16}$, each point correspondence contributes a homogeneous linear constraint of the form

$$\mathbf{A}_k h = 0,$$

and stacking multiple correspondences yields a linear system whose null space defines the candidate transformation. In the noise-free case, five point correspondences are sufficient to constrain a 15-DOF projective transform minimally. In practice, however, the dense predictions of feed-forward geometry models remain noisy, so directly solving the linear system from all points is not robust enough.

For this reason, robust estimation is usually applied on top of the DLT solver, for example by combining it with RANSAC in recent SLAM systems that optimize on the $SL(4)$ manifold [10]. Each iteration samples a minimal set of five correspondences, computes a candidate $\mathbf{H}$, and evaluates its support on the remaining points. The hypothesis with the best consensus set is then retained. This combination of a minimal linear solver and robust sampling is suitable for dense but imperfect correspondences produced by modern reconstruction networks.

The matrix recovered by DLT is only defined up to scale and therefore does not automatically satisfy the unit-determinant constraint of $SL(4)$. After selecting an orientation-preserving estimate with positive determinant, the matrix is normalized by

$$\mathbf{H}_{SL(4)} = \frac{\mathbf{H}_{\text{est}}}{\sqrt[4]{\det(\mathbf{H}_{\text{est}})}},$$

which enforces

$$\det(\mathbf{H}_{SL(4)}) = 1.$$

This projection step converts the estimated homography into a valid element of $SL(4)$ and allows subsequent optimization to remain on the intended manifold.

Compared with $SE(3)$ and $Sim(3)$ alignment, the main advantage of $SL(4)$ is its ability to absorb residual projective ambiguity before the scene has been fully metricized. The trade-off is that the transformation is less physically interpretable and less constrained. Therefore, $SL(4)$ is mainly useful in settings that operate in a projective or learned visual geometry space [10], whereas classical metric point cloud registration still primarily relies on $SE(3)$ or $Sim(3)$.

2.2.5 Robust Estimation with RANSAC

In practical scenes, data association inevitably contains outliers due to repetitive textures, motion blur, and dynamic objects. RANSAC (Random Sample Consensus) addresses this by repeatedly sampling minimal sets to hypothesize a model, scoring it by the number of inliers, and finally re-estimating the model using the best consensus set [11]. In visual SLAM, RANSAC is widely used for essential matrix estimation (e.g., the five-point method) and for PnP pose estimation [12, 13]. A typical iteration evaluates the geometric residual

$$\|\mathbf{x}' - \pi(\mathbf{T}\,\mathbf{X})\| < \tau,$$

where $\pi(\cdot)$ denotes the projection model, $\mathbf{T} \in SE(3)$ is the pose, and τ is an inlier threshold. Given an inlier ratio w and a minimal sample size s, the required number of iterations for confidence p is $N = \log(1-p)/\log(1-w^s)$ [11]. This procedure provides a robust front-end estimate that can be further refined by non-linear optimization.

2.2.6 Global Optimization

Global optimization refines the intermediate geometric estimates produced by front-end matching, triangulation, and tracking, with the goal of enforcing consistency over many views or long trajectories. In visual reconstruction and SLAM, two widely used formulations are bundle adjustment (BA), which optimizes poses and 3D points in the reprojection domain, and pose graph optimization (PGO), which optimizes only inter-pose constraints on a graph. Both belong to the back-end of SLAM/SfM systems and are commonly formulated as non-linear

least-squares problems, but they differ in their optimization variables and in the types of constraints they use.

Bundle Adjustment (BA)

Bundle adjustment jointly refines camera poses and 3D landmark positions by minimizing reprojection error [14]. Given camera poses $\{\mathbf{T}_i\}$ and landmarks $\{\mathbf{X}_j\}$, the classical objective is

$$\min_{\{\mathbf{T}_i,\mathbf{X}_j\}} \sum_{i,j} \rho\left(\|\mathbf{x}_{ij} - \pi(\mathbf{T}_i\,\mathbf{X}_j)\|^2\right),$$

where $\mathbf{x}_{ij}$ is the observed feature and $\rho(\cdot)$ is a robust loss. The optimization variables in BA usually include camera poses $\{\mathbf{T}_i\}$ and 3D points $\{\mathbf{X}_j\}$, and in some settings camera intrinsics may also be refined jointly. In essence, BA directly enforces that a reconstructed 3D point should project as closely as possible to the measured image observation. BA is typically solved with Levenberg–Marquardt using sparse linear algebra and the Schur complement to eliminate point variables efficiently [14, 15]. In practice, SfM often applies BA repeatedly as more images are added, while SLAM systems usually run local BA over a sliding window of recent keyframes to maintain real-time performance.

Pose Graph Optimization (PGO)

Pose graph optimization focuses on the global consistency of a trajectory after relative pose constraints have been estimated, for example from odometry edges and loop closures. In a typical pose graph, each node corresponds to a camera pose $\mathbf{T}_i$, while each edge encodes a relative motion constraint between two poses. Such constraints may arise from frame-to-frame odometry, loop-closure detection, ICP-based alignment, or relative transformations estimated between local submaps. Let the graph nodes be poses $\{\mathbf{T}_i\}$ and let each edge (i,j) provide a measured relative transformation $\hat{\mathbf{T}}_{ij}$. A common objective is

$$\min_{\{\mathbf{T}_i\}} \sum_{(i,j)\in\mathcal{E}} \left\|\mathrm{Log}\Big(\hat{\mathbf{T}}_{ij}^{-1}\left(\mathbf{T}_i^{-1}\mathbf{T}_j\right)\Big)\right\|_{\mathbf{\Omega}_{ij}}^2,$$

where $\mathrm{Log}(\cdot)$ maps the pose residual from $SE(3)$ to its tangent space and $\mathbf{\Omega}_{ij}$ is the information matrix of the constraint. In contrast to BA, there are no explicit 3D landmarks in this optimization: only poses are updated so that the relative constraints become globally consistent. Compared with BA, PGO is computationally cheaper because it optimizes only poses and omits explicit landmark variables. It is therefore widely used in SLAM back-ends to distribute loop-closure corrections over the full trajectory and to reduce accumulated drift, often with graph optimization frameworks such as g2o [16] and GTSAM [17].

Taken together, BA and PGO are closely related because both are graph-based back-end optimization methods and are often solved with similar non-linear least-squares frameworks. The main difference is that BA jointly optimizes motion and structure under reprojection constraints, whereas PGO optimizes only poses under relative transformation constraints.

For recent feed-forward neural reconstruction systems, however, the preference in the back-end often shifts from classical global BA to submap-level graph optimization. The reason is that the front-end no longer follows a traditional SfM pipeline built upon persistent feature tracks, explicitly maintained landmarks, and a clean reprojection-based measurement model. Instead, a neural network typically processes a short sequence jointly and directly predicts local depth, point clouds, poses, intrinsics, or confidence maps. Under this formulation, the front-end behaves more like a local reconstruction module that outputs a self-consistent submap, and the back-end is more naturally interpreted as aligning and stitching such submaps together.

This design choice is also motivated by efficiency and model structure. Since feed-forward reconstruction pipelines are already constrained by GPU memory, long sequences are commonly divided into multiple chunks or submaps. Performing global BA on top of these outputs would require jointly maintaining all camera poses, a large number of 3D points, and a massive set of reprojection residuals, which quickly becomes expensive. At the same time, many such systems do not provide a clean BA observation model with reliable long-term sparse tracks and clearly identified landmarks. In engineering practice, it is therefore more convenient to trust the network for local geometric recovery, estimate relative transformations between overlapping or loop-related submaps, and enforce global consistency through graph optimization. VGGT-SLAM follows this route by estimating inter-submap homographies from shared dense 3D points and optimizing them in a graph, rather than reformulating the full problem as a global pixel-level BA [10].

2.2.7 Depth-ray Representation

Recent learning-based reconstruction methods have also introduced new geometric parameterizations. In particular, DA3 proposes the depth–ray representation as a compact way to jointly encode scene geometry and camera motion without directly regressing constrained rotation matrices [18]. For each pixel $p = (u, v, 1)^\top$, the model predicts a depth value $D(u, v)$ together with a corresponding ray

$$r = (t, d), \quad d = RK^{-1}p,$$

where $t \in \mathbb{R}^3$ denotes the camera center and $d \in \mathbb{R}^3$ is the unnormalized ray direction in world coordinates.

The predicted ray map is denoted as

$$M \in \mathbb{R}^{H \times W \times 6},$$

where the first three channels $M(:, :, : 3)$ store per-pixel ray origins and the last three channels $M(:, :, 3 :)$ store the corresponding ray directions. Given the depth and ray predictions, a 3D point can be recovered by

$$P(u, v) = t + D(u, v) \cdot d.$$

Beyond 3D point reconstruction, the depth–ray representation also enables direct recovery of camera parameters. The camera center t_c is estimated by averaging the per-pixel ray origins:

$$t_c = \frac{1}{HW} \sum_{h=1}^{H} \sum_{w=1}^{W} M(h, w, : 3).$$

To recover the camera rotation R and intrinsic matrix K, DA3 formulates the problem as estimating a homography. An identity canonical camera is first defined with intrinsics $K_I = I$. For a pixel p, the corresponding ray direction in this canonical camera is

$$d_I = K_I^{-1}p = p.$$

The transformation from the canonical ray d_I to the ray direction d_{cam} in the target camera coordinate system can be written as

$$d_{\text{cam}} = KRd_I,$$

which establishes a homography relationship

$$H = KR.$$

The homography H is then estimated by minimizing the geometric error between transformed canonical rays and the predicted ray directions:

$$H^* = \arg\min_{\|H\|=1} \sum_{h=1}^{H} \sum_{w=1}^{W} \|Hp_{h,w} \times M(h, w, 3:)\|.$$

This optimization corresponds to a standard least-squares problem and can be efficiently solved using the Direct Linear Transform (DLT) algorithm. Once the optimal homography H^* is obtained, the camera intrinsics K and rotation R are uniquely recovered via RQ decomposition, exploiting the upper-triangular structure of K and the orthonormality of R.

By jointly modeling depth and rays, this representation avoids explicit pose constraints while ensuring geometric consistency across views, and provides a compact yet expressive abstraction for feed-forward 3D reconstruction.

2.2.8 SLAM

Building upon the geometric representations and optimization tools introduced above, Visual Simultaneous Localization and Mapping (SLAM) combines these components into a complete online system that estimates the camera trajectory while incrementally constructing a map of the environment from image streams. A comprehensive survey of the field can be found in [19]. Classical visual SLAM systems are often keyframe-based and feature-driven, e.g., PTAM [20] and ORB-SLAM/ORB-SLAM2 [21, 22]. In contrast, direct methods such as LSD-SLAM [23] and DSO [24] avoid explicit feature matching and minimize photometric error over selected pixels. These two families share a common structure: a front-end responsible

for data association and pose tracking, and a back-end that enforces global consistency via optimization.

Standard Visual SLAM Pipeline

Despite differences in sensors and implementations, most classical visual SLAM systems follow a modular pipeline that separates tracking, mapping, and optimization [19, 21, 22], as illustrated in Fig. 2.5.

1. **Initialization:** the system is bootstrapped by estimating an initial set of camera poses and map points from a short image sequence, typically using monocular epipolar geometry or direct depth measurements in stereo/RGB-D settings.

2. **Tracking (front-end):** for each incoming frame, visual features (e.g., ORB keypoints) are extracted and matched against the local map. The current camera pose is estimated by solving a Perspective-n-Point (PnP) or epipolar geometry problem, with robust outlier rejection commonly performed using RANSAC to handle incorrect correspondences.

3. **Local mapping:** selected keyframes are inserted into the map, new 3D landmarks are triangulated, and redundant points or keyframes are culled. To maintain local consistency, a local bundle adjustment (BA) is applied to jointly refine recent camera poses and map points.

4. **Loop closure and global optimization:** revisited places are detected using visual place recognition. Once a loop is confirmed, global consistency is enforced by pose graph optimization or global bundle adjustment, correcting accumulated drift in camera poses and the map.

This clear separation between a fast front-end for real-time pose tracking and a back-end for global optimization has been a defining characteristic of classical visual SLAM systems.

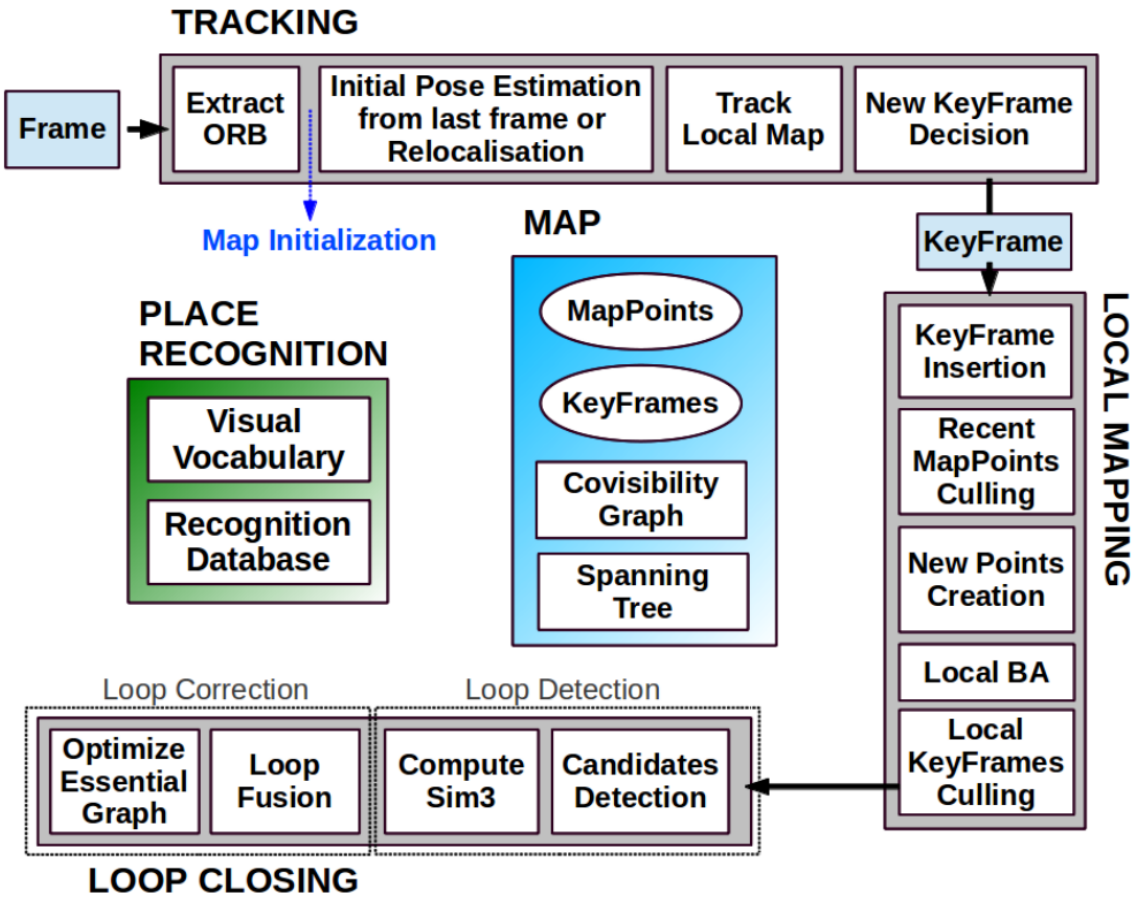

Figure 2.5: Standard visual SLAM pipeline

2.3 Learning-Based Dense 3D Reconstruction

This section introduces the learning-based dense reconstruction methods that are most relevant to this thesis. It first summarizes the transformer-based backbone and dense prediction modules used in modern geometry networks, and then reviews the transition from early learned components to recent feed-forward multi-view reconstruction models, culminating in the DA3 model adopted in this work.

2.3.1 Vision Transformer (ViT)

Vision Transformer (ViT) transfers the standard Transformer architecture from natural language processing to visual recognition by representing an image as a sequence of fixed-size patches rather than as a grid processed by convolutions.[25] Given an input image of size $H \times W \times C$, the image is split into $N = \frac{HW}{P^2}$ non-overlapping patches of size $P \times P$. Each patch is flattened and linearly projected into a D-dimensional embedding. A learnable class token is prepended to the patch sequence, and positional embeddings are added to preserve spatial order before the tokens are passed through stacked Transformer encoder blocks.

Each Transformer block consists of multi-head self-attention and a feed-forward multilayer perceptron, together with residual connections and layer normalization.[25] In contrast to convolutional neural networks, ViT does not impose strong image-specific inductive biases such as locality or translation equivariance. Instead, it learns spatial relationships through global self-attention, allowing each patch token to directly interact with all others. This makes ViT especially suitable for capturing long-range dependencies and globally consistent structures across the image, although it also makes the model more dependent on large-scale pretraining data.

For the present work, ViT is relevant because it serves as the architectural basis of several recent feed-forward geometry models. Its token-based formulation is naturally compatible with cross-view reasoning, and the globally contextualized features it produces are well suited for dense reconstruction tasks, where geometric correspondences and scene structure often extend over large image regions rather than remaining confined to local neighborhoods.

2.3.2 Self-Distillation with No Labels (DINO)

DINO is a self-supervised representation learning framework that learns visual features without manual annotations by distilling the outputs of a teacher network into a student network of the same architecture.[26] In its standard form, both teacher and student are Vision Transformers. For each input image, multiple augmented views are generated. The student processes all views, while the teacher usually receives only the global crops. Training minimizes a cross-entropy loss between the output distributions of the teacher and the student, with gradients stopped on the teacher branch.

The teacher network is not fixed beforehand. Instead, its parameters are updated as an expo-

nential moving average of the student, which stabilizes training.[26] To avoid representational collapse, DINO applies centering and sharpening to the teacher outputs. It also benefits strongly from multi-crop augmentation, where local and global image crops provide complementary supervision. This design avoids the need for labeled data, negative sample mining, or large memory banks while still producing robust and transferable features.

An important observation reported in DINO is that self-supervised ViT features can encode meaningful object-level structure and often reveal salient regions and boundaries in the scene.[26] Such properties are valuable for geometric vision because dense reconstruction does not only require semantic robustness, but also spatially coherent and structurally informative features. In the context of DA3, DINO is particularly relevant because the model relies on a DINO-pretrained transformer backbone to provide general-purpose visual representations before the geometry-specific prediction heads are applied.

2.3.3 Dense Prediction Transformer (DPT)

While ViT was originally proposed for image-level tasks such as classification, dense prediction tasks require high-resolution outputs for every pixel. Dense Prediction Transformer (DPT) addresses this gap by adapting transformer features to dense prediction problems such as monocular depth estimation and semantic segmentation.[27] Instead of relying on a convolutional backbone that progressively downsamples the image, DPT uses a ViT encoder that maintains a relatively high and constant feature resolution throughout the encoding process while providing a global receptive field at every stage.

A key idea in DPT is the reassembly of transformer tokens into image-like feature maps at multiple resolutions.[27] Features extracted from several transformer layers are reshaped and projected back into spatial representations, after which fusion modules progressively combine them in a convolutional decoder. In this way, DPT integrates the global contextual reasoning of the transformer encoder with the local refinement capability of dense decoders, yielding predictions that are both globally coherent and locally detailed.

For this thesis, DPT is important because it provides the architectural template that connects transformer backbones to dense geometric outputs. Modern feed-forward reconstruction models, including the ones discussed later in this chapter, use DPT-style heads to decode shared visual features into depth-related quantities. Therefore, understanding DPT helps explain how high-level transformer representations are converted into the dense depth, point, or ray

predictions required for 3D reconstruction.

2.3.4 Feed-Forward Transformer-Based 3D Reconstruction

Early learning methods injected robustness at the component level: learned detectors [20], descriptors for matching [22], and differentiable optimization layers that expose pose/depth updates to gradient flow [31, 33, 62]. On the dense side, cost-volume networks [106, 114] for MVS replaced hand-crafted regularization with 3D CNNs, improving depth accuracy especially at large baselines and thin structures compared with classical PatchMatch.[18]

They did reduce engineering complexity and demonstrated the feasibility of learned joint depth pose estimation, but they often struggled with scalability, generalization, and handling arbitrary input cardinalities.[18]

Traditional geometry-based methods remain strong on well-textured scenes, but their modularity and brittle correspondences complicate robustness under low texture, specularities, or large viewpoint changes.

The emergence of feed-forward 3D reconstruction methods is closely tied to the introduction of attention mechanisms. Unlike convolutional operations that rely on local receptive fields, attention enables explicit modeling of long-range dependencies by computing pairwise interactions between elements in a sequence. In the context of multi-view geometry, this property is particularly appealing, as 3D reconstruction inherently requires reasoning over global spatial relationships and correspondences across views. By allowing features from different images to directly attend to each other, attention mechanisms provide a natural alternative to explicit feature matching and geometric verification used in traditional pipelines.

Leveraging attention-based architectures, recent approaches formulate 3D reconstruction as a feed-forward prediction problem, where geometric quantities such as depth, camera motion, or point maps are directly inferred from image features. This paradigm eliminates the need for iterative optimization and decoupled processing stages, enabling end-to-end learning of visual geometry from data.

DUSt3R: A Pioneering Feed-forward Transformer-based Method

A turning point came with DUSt3R,a radically novel paradigm for Dense and Unconstrained Stereo 3D Reconstruction of arbitrary image[28], which leveraged transformers to directly predict point map between two views and compute both depth and relative pose in a purely feed-forward manner. This work laid the foundation for subsequent transformer-based methods aiming to unify multi-view geometry estimation at scale.[18]

As illustrated in Fig. 2.6, given two input images I_1 and I_2, both images are first processed by a shared image feature extractor to obtain dense visual representations. The feature extractor can be implemented using either convolutional neural networks or transformer-based architectures; in DUSt3R, a Vision Transformer (ViT) is adopted as the encoder, whose architectural principles have been introduced above. The extracted feature tokens from the two views are then passed to two transformer decoders, which iteratively exchange information through cross-attention. This mechanism allows features from one view to attend to and reason about features from the other view, enabling implicit correspondence discovery without explicit feature matching.

Based on the fused multi-view features, two regression heads predict dense point maps for each input image, along with associated confidence maps. Importantly, both point maps are expressed in a common coordinate frame defined by the first camera view I_1, thereby establishing a shared geometric reference across the two images. From the predicted point maps, camera intrinsics such as focal lengths can be estimated using the Weiszfeld algorithm [29], and relative camera poses can be recovered using minimal solvers such as 3-point RANSAC [12, 13].

By directly regressing dense geometry in a feed-forward manner, DUSt3R eliminates the need for explicit feature matching, geometric verification, and iterative optimization, demonstrating the feasibility of learning-based geometric reasoning from image pairs. [10]

Follow-up Works

MASt3R follows a similar design but also outputs descriptors that can be used to generate pairwise correspondences between the two frames. MASt3R-SFM [14] demonstrates global optimization of multiple images using MASt3R but computation scales quickly with the number of frames. [10]

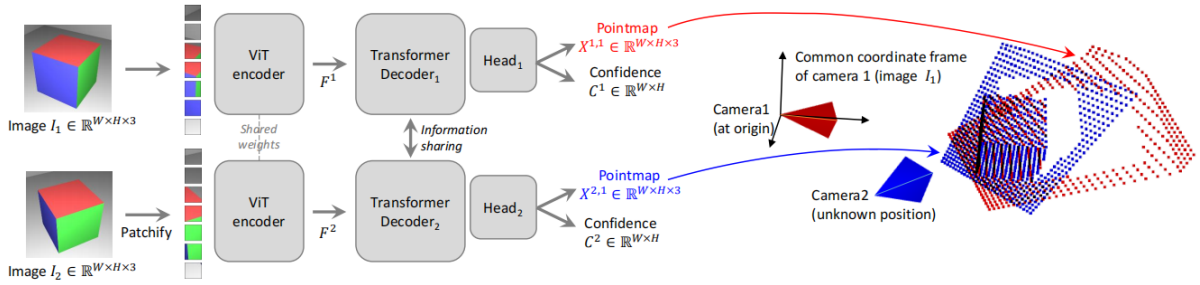

Figure 2.6: DUSt3R framework overview

To extend the idea of DUSt3R to multiple frames, Spann3R [67] leverages a learned memory module and Cut3R [69] uses a recurrent state model. Both can incrementally reconstruct a scene using multiple images, but are each limited to short sequences. Recently, Pow3R [27] extends the DUSt3R framework to optionally take in any estimates of any combination of camera intrinsics, poses, and depth (which may be sparse or dense) and demonstrates substantial improvement in scene reconstruction and pose estimation given the added inputs. Splatt3R [60] extends the DUSt3R idea to Gaussian Splatting [29] by directly outputting the Gaussian Splatting parameters given two views, and PreF3R [8] extends this to multiple views using a similar memory framework as Spann3R. Reloc3r [13] modifies the DUSt3R framework for directly outputs relative camera poses and uses motion averaging to recover absolute poses with respect to a map database.[10]

Visual Geometry Grounded Transformer

While DUSt3R and its follow-up works demonstrate that feed-forward models can successfully infer dense geometry from image pairs, their pairwise formulation inherently limits scalability and global reasoning. Extending such methods to multi-view settings typically requires additional post-processing steps, such as pose graph optimization or incremental fusion, which reintroduce elements of traditional pipelines.

These limitations motivate the need for a unified framework that can jointly reason over an arbitrary number of views and directly model global geometric relationships in a feed-forward manner.

As a culmination of feed-forward reconstruction models inspired by DUSt3R, VGGT (Visual Geometry Grounded Transformer) extends pairwise geometric reasoning to arbitrary-length image sequences within a unified architecture. An overview of the VGGT framework is shown

in Figure 2.7. Given a set of input images, VGGT first decomposes each image into a sequence of visual tokens using a pretrained image feature extractor. In the original formulation, self-supervised features obtained from DINO are employed to provide semantically rich and geometrically consistent representations. These per-image token sequences are then concatenated, and camera-specific tokens are appended to enable explicit modeling of camera parameters.

The core of VGGT consists of multiple transformer layers that alternate between global attention and frame-wise attention. Global attention allows tokens from different views to directly interact, facilitating cross-view information exchange and long-range geometric reasoning. In contrast, frame-wise attention focuses on refining features within each individual image. By alternating between these two attention mechanisms, VGGT jointly captures both intra-view structure and inter-view geometric relationships across the entire image set.

Based on the aggregated token representations, VGGT employs multiple prediction heads to regress diverse geometric quantities. A dedicated camera head estimates camera intrinsics and extrinsics, while dense prediction transformer (DPT) heads are used to generate dense depth maps, point maps, and feature tracks. Importantly, these outputs are expressed in a common coordinate frame, enabling consistent multi-view reconstruction without explicit correspondence estimation or incremental optimization.

By unifying multi-view feature aggregation and geometry prediction within a single feed-forward transformer, VGGT represents a significant step toward holistic and scalable dense 3D reconstruction from unconstrained image collections.

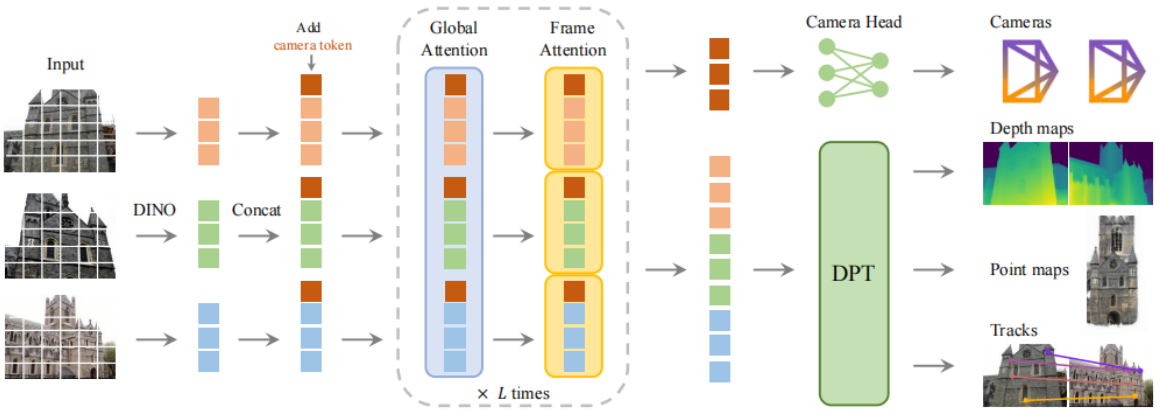

Figure 2.7: VGGT framework overview

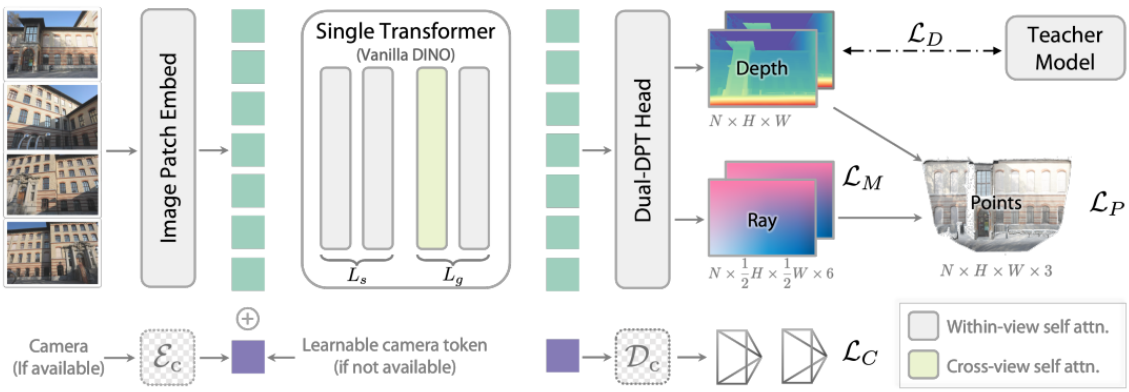

Figure 2.8: DA3 framework overview

DepthAnything3

While VGGT represents a significant advancement in unified feed-forward multi-view reconstruction, it still leaves room for improvement in terms of efficiency, scalability, and generalization. The VGGT framework is characterized by a large and complex architecture, with model sizes reaching up to one billion parameters, which makes it unsuitable for lightweight deployment scenarios. Although VGGT supports arbitrary-length input sequences, this flexibility comes at the cost of substantial memory consumption and computational overhead, especially when processing long image sequences. Moreover, despite being trained on large-scale and diverse datasets covering a wide range of common visual scenes, VGGT may still fail when confronted with domains that differ significantly from its training distribution, such as endoscopic imagery. These limitations highlight the need for a more compact and robust reconstruction model that preserves the strengths of VGGT while addressing its practical shortcomings. To this end, Depth Anything 3 (DA3) is introduced as an improved variant built upon the VGGT paradigm. Building upon this insight, DA3 proposes a unified feed-forward model that reconstructs the visual space from arbitrary visual inputs, including multi-view image collections and video streams. Given any number of input views, with or without known camera poses, DA3 jointly predicts dense depth maps and ray maps that can be directly fused into geometrically consistent 3D point clouds. The depth–ray representation proposed by DA3 provides an important geometric abstraction and has therefore been introduced earlier in Section 2.2.7. The focus here is on the overall model design and training strategy.

Compared to VGGT, DA3 significantly simplifies the overall framework by relying on a single DINO-based transformer backbone and enabling cross-view reasoning through token rearrangement rather than explicit multi-branch architectures. This design reduces the model size to

a configurable range between 0.11B and 0.35B parameters, making DA3 substantially more suitable for lightweight and resource-constrained deployment while maintaining strong reconstruction performance. As illustrated in Fig. 2.8, the model takes an arbitrary set of input views and processes them with a shared transformer backbone.

Cross-view geometric reasoning is achieved through an input-adaptive self-attention mechanism. Instead of introducing additional transformer stacks or specialized modules, DA3 rearranges token sequences in selected layers, allowing standard self-attention to operate either within individual views or across all views jointly.

Dense geometric predictions in DA3 are produced by a novel dual Dense Prediction Transformer (Dual-DPT) head, which jointly regresses dense depth maps and ray maps from a shared set of visual features. As illustrated in Fig. 2.9, the Dual-DPT head first processes backbone features through a common set of reassembly modules, ensuring that both prediction tasks operate on a unified and compact intermediate representation.

After this shared processing stage, the features are routed into two parallel branches, corresponding to depth prediction and ray prediction, respectively. Each branch applies its own set of fusion layers to aggregate contextual information relevant to the target geometric quantity. Finally, separate output layers generate the depth map and ray map predictions. By sharing the majority of the feature processing pipeline and diverging only at the final fusion stage, the Dual-DPT design promotes strong interaction between depth and ray estimation while avoiding redundant intermediate representations.

In addition to dense geometric outputs, DA3 optionally predicts camera parameters through a lightweight camera head operating on dedicated camera tokens, incurring minimal computational overhead. This joint prediction framework further reinforces geometric consistency without compromising the overall efficiency of the model.

To train a generalist geometry model across diverse data sources, DA3 adopts a teacher–student learning paradigm. A teacher model is first trained on large-scale synthetic data to generate dense and reliable pseudo-depth supervision, which is then combined with sparse or noisy real-world depth measurements. This strategy enables effective knowledge transfer while preserving geometric fidelity across diverse domains.

Formally, the DA3 model F_θ maps an input image set $\mathcal{I}$ to a set of geometric predictions, including a depth map $\hat{D}$, a ray map $\hat{R}$, and an optional camera pose $\hat{c}$:

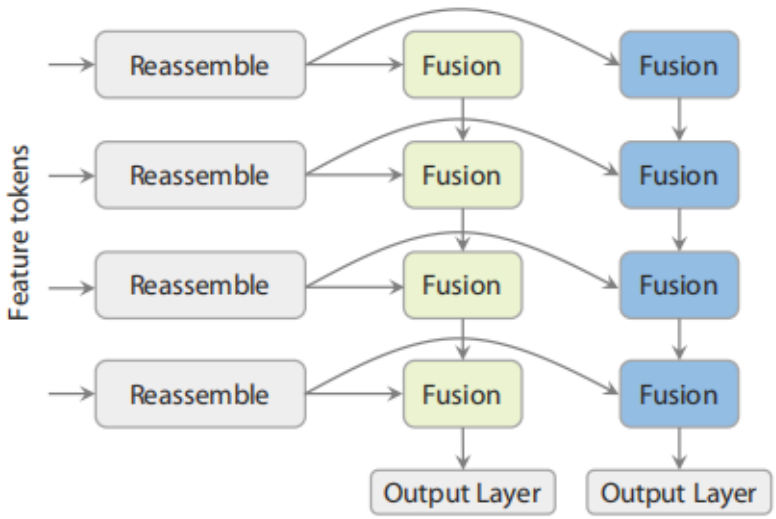

Figure 2.9: DA3 Dual-DPT Head

$$F_\theta(\mathcal{I}) \to \{\hat{D}, \hat{R}, \hat{c}\}.$$

The camera pose prediction is optional and primarily included for practical convenience.

Prior to loss computation, all ground-truth geometric signals are normalized by a common scale factor to ensure consistent magnitude across different modalities. This scale is defined as the mean ℓ_2 norm of the valid reprojected 3D point maps P, which stabilizes optimization when jointly supervising depth, rays, and reconstructed points.

The overall training objective is defined as a weighted sum of several loss terms:

$$L = L_D(\hat{D}, D) + L_M(\hat{R}, M) + L_P(\hat{D} \odot d + t, P) + \beta L_C(\hat{c}, c) + \alpha L_{\text{grad}}(\hat{D}, D),$$

where L_D supervises depth prediction, L_M enforces consistency of ray maps, L_P penalizes geometric errors in reconstructed 3D points, and L_C is an optional camera pose loss.

The depth supervision term is defined as

$$L_D(\hat{D}, D; D_c) = \frac{1}{|\Omega|} \sum_{p \in \Omega} m_p (D_{c,p} |\hat{D}_p - D_p| - \lambda_c \log D_{c,p}),$$

where $D_{c,p}$ denotes the confidence associated with the ground-truth depth D_p, m_p is a validity mask, and Ω denotes the set of valid pixels. All loss terms are based on the ℓ_1 norm.

To further regularize the depth predictions, a gradient loss is introduced:

$$L_{\text{grad}}(\hat{D}, D) = \|\nabla_x \hat{D} - \nabla_x D\|_1 + \|\nabla_y \hat{D} - \nabla_y D\|_1,$$

where ∇_x and ∇_y denote horizontal and vertical finite difference operators. This term preserves sharp depth discontinuities while encouraging smoothness in planar regions. In practice, the weighting factors are set to $\alpha = 1$ and $\beta = 1$.

This training strategy enables DA3 to effectively leverage heterogeneous datasets while maintaining consistent geometric supervision, resulting in a scalable and robust feed-forward reconstruction model.

2.4 State of the Art

This section reviews the state of the art most relevant to the present work, which combines dense reconstruction of endoscopic scenes with real-time localization and mapping. In endoscopic settings, both tasks are significantly harder than in standard outdoor or rigid indoor scenes because the visual input often contains specular highlights, low-texture tissue surfaces, repeated patterns, rapid viewpoint changes, and local non-rigid deformations. As a result, methods must balance geometric accuracy, robustness, and computational efficiency much more carefully than in conventional vision pipelines.

Accordingly, Table 2.1 is organized into two groups. The first group summarizes dense endoscopic reconstruction methods, including both endoscopy-specific approaches and recent general reconstruction foundation models that are relevant as transferable backbones. The second group focuses on endoscopic SLAM systems that jointly address camera tracking and online map building. Methods that are mainly generic fusion backbones without direct endoscopic validation are omitted here to keep the comparison aligned with the problem setting of this thesis.

Table 2.1: Overview of State-of-the-Art Methods for Endoscopic Dense Reconstruction and Endoscopic SLAM

Source	Goal	Methods	Key Results
		Dense 3D Reconstruction	

Source	Goal	Methods	Key Results
Endo3R [30]	Online and scale-consistent dense reconstruction from monocular endoscopic video	Feed-forward reconstruction foundation model with pointmap, depth, and camera prediction; uses a dual-memory design for long video streams.	Shows strong zero-shot depth and pose estimation with online reconstruction, supporting feed-forward foundation models for surgical videos.
EndoDAC [31]	Self-supervised monocular depth estimation for downstream endoscopic reconstruction	Foundation-model adaptation of Depth Anything with DV-LoRA, convolutional neck blocks, and joint pose–intrinsics prediction.	Improves self-supervised endoscopic depth estimation with only about 1.6M trainable parameters and without requiring known camera intrinsics.
SfMDiffusion [32]	Self-supervised monocular depth estimation for downstream endoscopic reconstruction	Diffusion-based depth estimation with teacher–student distillation, pseudo-ground-truth depth diffusion, and discriminative priors.	Achieves strong depth accuracy on SCARED and Hamlyn, e.g. Abs Rel 0.049 and RMSE 4.305 on SCARED, showing the benefit of diffusion priors in surgical scenes.
Endo3DAC [33]	Self-supervised endoscopic 3D scene reconstruction from arbitrary cameras	Foundation-model adaptation with a frozen ViT backbone, GDV-LoRA modules, and an integrated depth–pose–intrinsics network.	Outperforms earlier self-supervised endoscopic baselines across four datasets while using only surgical videos and about 10.2M trainable parameters.
DA3 [18]	General feed-forward multi-view geometry recovery from arbitrary views	Feed-forward transformer geometry model with a ViT backbone and Dual-DPT depth–ray prediction head.	Delivers strong dense geometry and camera recovery in a single forward pass, making it a strong transferable reconstruction baseline.

Source	Goal	Methods	Key Results
VGGT [6]	Unified visual geometry prediction across multi-view image sets	Feed-forward transformer foundation model that jointly predicts cameras and dense structure in a projective-consistent visual space.	Demonstrates strong general multi-view geometry reasoning.
Real-Time Dense Reconstruction with Binocular Endoscopy [34]	Dense metric reconstruction with stronger geometric scale control in endoscopic scenes	Classical stereo depth plus ORB-SLAM and point-cloud fusion in an RGB-D-style pipeline.	Achieves real-time metric reconstruction and denser point clouds
SLAM			
OneSLAM [35]	General monocular SLAM for multiple endoscopic domains without retraining	Monocular SLAM built on TAP-based sparse correspondence tracking and local bundle adjustment for joint pose and sparse map refinement.	Achieves better or comparable performance, showing strong cross-domain generalization out of the box.
EndoGSLAM [36]	Real-time endoscopic tracking and dense tissue reconstruction for intraoperative use	Monocular dense SLAM with online camera pose optimization and differentiable rendering-based map updates for real-time tracking.	Achieves a strong speed–quality trade-off, with the Gaussian representation enabling dense reconstruction and rendering above 100 fps.
Magnetic-Visdeual Sensor Fusion for Capsule Endoscopy [37]	Robust localization and dense reconstruction when visual information becomes unreliable	Multi-sensor SLAM that fuses magnetic tracking with endoscopic vision.	Demonstrates that external sensing can substantially reduce drift under low texture and occlusion, at the cost of additional hardware.

3 Materials and Methods

This chapter presents the methodological design of the proposed system in two stages. First, we introduce DynaDA3 as the dense reconstruction module, which adapts the DA3 framework to endoscopic scenes and enhances the geometric prediction with dynamic occlusion filtering. Building upon this reconstruction module, we then present DynaDA3-SLAM, which extends DynaDA3 with SLAM functionality to support real-time localization, mapping, and navigation in monocular surgical environments. Figure 3.1 provides an overview of the DynaDA3 architecture, while Figure 3.2 summarizes the overall DynaDA3-SLAM pipeline.

3.1 DynaDA3

3.1.1 Preprocessing

Endoscopic video data typically presents unique visual challenges compared to standard datasets, including a circular field of view (FOV) surrounded by black borders and non-uniform illumination characterized by specular reflections and underexp§osed peripheral regions. To ensure the robustness of the subsequent network inference, we apply a two-stage preprocessing pipeline comprising region of interest cropping and illumination correction.

Region of Interest Cropping

To eliminate irrelevant information from the black borders and focus the network's attention on the surgical site, we employ a fixed-ratio cropping strategy. As implemented in our pipeline, we extract a square region from the original video frames. The crop size is determined dynamically based on the image height:

$$S_{crop} = H_{img} \times \lambda_{crop}$$

where H_{img} is the height of the original frame and λ_{crop} is a scaling factor (experimentally set to 0.65 for the C3VD2 dataset) to cover the relevant endoscopic view. The crop window is centered horizontally with a learned offset to align with the endoscopic center, and positioned vertically to retain a balanced view of the surgical cavity, discarding the extreme upper and lower black margins.

Illumination Correction

To reduce the impact of severe specular highlights in endoscopic images, we adopt a learning-based two-stage highlight suppression strategy inspired by the method of Li et al. [38]. In contrast to heuristic brightness adjustment, this approach explicitly detects highlight regions and then restores the corrupted appearance using context-aware inpainting, which is more suitable for surgical scenes where saturated reflections frequently obscure tissue texture.

In the first stage, the cropped RGB frame is processed by a Hierarchical Feature Attention Network (HFA-Net), which predicts a binary highlight mask $\mathbf{M}$ together with a coarse de-highlighted image $\mathbf{D}_{\text{coarse}}$. Following the improved dichromatic reflection formulation in [38], the coarse image is modeled as

$$\mathbf{D}_{\text{coarse}} = \mathbf{I} \odot (1 - \mathbf{M}) + (\mathbf{I} - \mathbf{S}) \odot \mathbf{M},$$

where $\mathbf{I}$ denotes the input image, $\mathbf{S}$ is the estimated specular component, and $\odot$ denotes element-wise multiplication. This stage focuses on accurate localization of highlight regions and an initial suppression of saturated reflections.

The HFA-Net backbone combines hierarchical attention and local feature refinement to better distinguish true specular highlights from naturally bright tissue regions. In particular, the network integrates spatial-shift segmented attention, dual-flow attention, multi-scale feature fusion, and partial mask convolution, enabling both precise highlight detection and coarse removal [38]. This is important in endoscopic imagery, where simple thresholding often confuses wet tissue reflections with normal bright anatomical structures.

In the second stage, the coarse result is refined using a large-mask inpainting model based on LaMa. Before inpainting, the predicted highlight mask is morphologically dilated to enlarge the

corrupted region and expose a wider contextual neighborhood. The dilated mask together with $\mathbf{D}_{\text{coarse}}$ are then fed into the inpainting network, which reconstructs texture-consistent tissue appearance and produces smoother transitions between corrected and unmodified regions. Compared with direct filtering, this two-stage design better preserves structural continuity around highlight boundaries and reduces visible artifacts in restored areas [38].

The final output of this preprocessing step is a highlight-suppressed RGB image that is used as the input to the DynaDA3 network. By reducing saturated reflections before geometric inference, this step improves the visibility of tissue texture and helps stabilize subsequent depth, ray, and confidence predictions.

3.1.2 Architecture

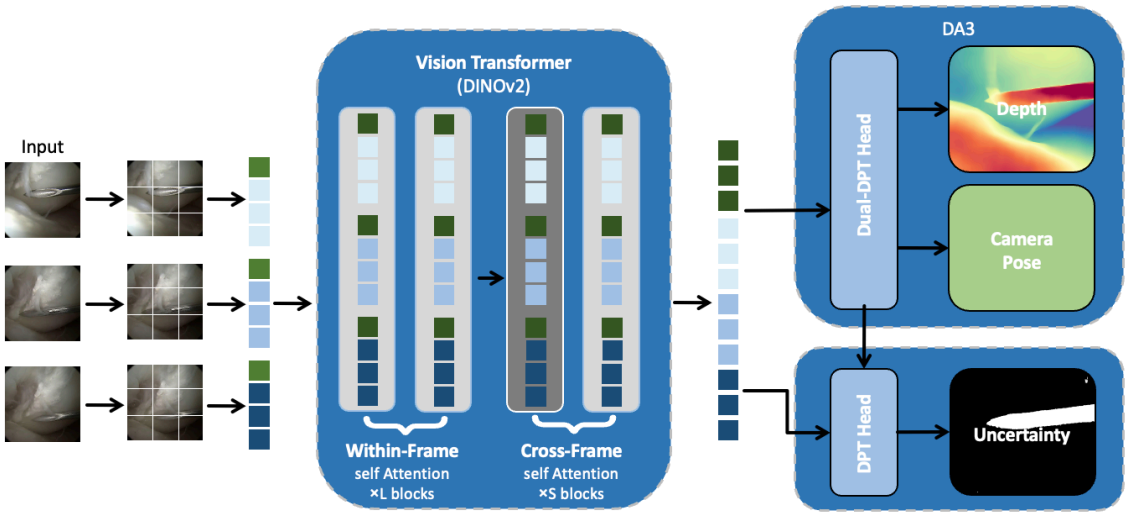

Figure 3.1: Overview of the proposed DynaDA3 architecture. The pipeline combines preprocessing, frozen DA3-based geometric prediction, and a trainable DPT head for dynamic occlusion filtering.

As illustrated in Figure 3.1, the DynaDA3 architecture is built on top of the Depth Anything V3 (DA3) framework [18], but it does not retrain the DA3 foundation model itself. Instead, the pretrained DA3 module is kept frozen and reused as a geometric prior extractor, while a newly introduced uncertainty-aware DPT head is trained on top of DA3 outputs. For an input batch of multi-frame images $\mathbf{I} \in \mathbb{R}^{B\times N\times 3\times H\times W}$, the frozen DA3 stage first produces dense depth, per-pixel depth confidence, and four intermediate transformer feature maps. These signals are then fused by a trainable `UncertaintyDPT` head to predict a two-class segmentation logit map, where class 0 denotes certain or static regions and class 1 denotes uncertain or dynamic

regions. In this way, DynaDA3 preserves the broad geometric generalization capability of DA3 while specializing only the final uncertainty filtering stage for dynamic surgical environments.

DA3 Backbone and Geometric Outputs

The frozen DA3 stage follows the original modular design of a DINOv2 backbone, a Dual-DPT dense prediction head, and an optional camera branch [18]. In the implementation used here, the large DA3 variant uses a ViT-L backbone and exports intermediate features from transformer layers $\{11, 15, 19, 23\}$, while the giant variant uses a ViT-G backbone with layers $\{21, 27, 33, 39\}$. During training, `DynaDA3.forward()` directly calls `self.da3.model(image, export_feat_layers=...)` under `torch.no_grad()`, so the DA3 stage remains strictly frozen. The returned dictionary contains four exported feature tensors stored in `out['aux']` under keys of the form `feat_layer_<idx>`, each with shape $\mathbb{R}^{B\times N\times h\times w\times C}$, a dense depth map `out['depth']` $\in \mathbb{R}^{B\times N\times H\times W}$, and a dense confidence map `out['depth_conf']` $\in \mathbb{R}^{B\times N\times H\times W}$.

Before entering the uncertainty head, the wrapper explicitly reshapes these DA3 outputs. For each exported layer ℓ_k, the feature tensor is converted from

$$\mathbf{F}^{(\ell_k)} \in \mathbb{R}^{B\times N\times h\times w\times C}$$

to

$$\tilde{\mathbf{F}}^{(\ell_k)} \in \mathbb{R}^{(BN)\times C\times h\times w},$$

while the depth and confidence maps are reshaped from $\mathbb{R}^{B\times N\times H\times W}$ to $\mathbb{R}^{(BN)\times 1\times H\times W}$. Thus, the uncertainty head does not operate on temporally aggregated tokens; instead, it processes all sampled frames independently after flattening the batch and frame dimensions into a single dimension $M = BN$. In the inference path, the same information is obtained through `self.da3.inference(...)` and then read from `prediction.aux`, `prediction.depth`, and `prediction.conf`. Therefore, in both training and inference, the uncertainty head receives the same three signal groups: four DA3 feature maps, one depth map, and one confidence map per frame.

Keeping the complete DA3 stage frozen is a deliberate design choice. The goal of DynaDA3 is not to relearn generic geometry from limited task-specific data, but to reuse DA3 as a

stable geometric foundation and to add a specialized uncertainty segmentation layer on top. This strategy reduces the number of trainable parameters substantially and helps preserve the robustness of the pretrained model in out-of-domain scenes.

Uncertainty-Aware DPT Head

Although named `UncertaintyDPT`, the current implementation is a lightweight convolutional fusion head operating on DA3 features rather than a full multi-stage decoder. Let $M = BN$ denote the flattened frame dimension, let the legal input resolution satisfy $H = ph$ and $W = pw$ with patch size $p = 14$, and let the embedding dimension be $C_e = 2048$. The four DA3 feature maps are first projected independently by 1×1 convolutions:

$$\mathbf{P}_k = \phi_k\left(\tilde{\mathbf{F}}^{(\ell_k)}\right) = \mathrm{ReLU}\left(\mathrm{Conv}_k^{1\times 1}\left(\tilde{\mathbf{F}}^{(\ell_k)}\right)\right), \qquad \mathbf{P}_k \in \mathbb{R}^{M\times C_e\times h\times w},$$

and then summed:

$$\mathbf{P}_{img} = \sum_{k=1}^{4} \mathbf{P}_k.$$

This part corresponds exactly to the four `projects_layer` modules in the code.

The geometric branch is constructed explicitly from the DA3 depth and confidence outputs. First, both maps are normalized frame-wise by robust percentile normalization. For a scalar map $\mathbf{X} \in \mathbb{R}^{M\times 1\times H\times W}$, the code computes

$$q_{low}^{(m)} = Q_{0.05}\left(\mathbf{X}^{(m)}\right), \qquad q_{high}^{(m)} = Q_{0.95}\left(\mathbf{X}^{(m)}\right),$$

and then

$$\bar{\mathbf{X}}^{(m)} = \mathrm{clip}\left(\frac{\mathbf{X}^{(m)} - q_{low}^{(m)}}{\max\left(q_{high}^{(m)} - q_{low}^{(m)}, \varepsilon\right)}, 0, 1\right), \qquad \varepsilon = 10^{-6},$$

where $Q_{0.05}$ and $Q_{0.95}$ denote the 5th and 95th percentiles, respectively. Applying this operation to depth and confidence yields normalized maps $\bar{\mathbf{D}}$ and $\bar{\mathbf{C}}$. This design is directly taken

from the helper function `_geo_normalize()` and prevents extreme outliers from dominating the feature scale.

Next, a depth-gradient map is computed by first-order forward differences:

$$\mathbf{G}_x(u,v) = \left|\bar{\mathbf{D}}(u,v+1) - \bar{\mathbf{D}}(u,v)\right|, \qquad \mathbf{G}_y(u,v) = \left|\bar{\mathbf{D}}(u+1,v) - \bar{\mathbf{D}}(u,v)\right|,$$

and the final gradient prior is formed as

$$\mathbf{G} = \mathrm{Pad}_x(\mathbf{G}_x) + \mathrm{Pad}_y(\mathbf{G}_y),$$

where zero padding restores the original spatial size along the last column and last row. In code, the geometric priors are then transferred from image space to patch space by `pixel_unshuffle` with factor $p = 14$. For a generic tensor $\mathbf{X} \in \mathbb{R}^{M\times C\times H\times W}$, this operation can be written as

$$\mathrm{PU}(\mathbf{X})_{m,\, cp^2+ip+j,\, a,\, b} = \mathbf{X}_{m,\, c,\, pa+i,\, pb+j},$$

with $i, j \in \{0, \ldots, p-1\}$, $a \in \{0, \ldots, h-1\}$, and $b \in \{0, \ldots, w-1\}$. Thus, each 14×14 local patch is folded into the channel dimension.

Defining

$$\tilde{\mathbf{D}} = \mathrm{PU}(\bar{\mathbf{D}}), \qquad \tilde{\mathbf{C}} = \mathrm{PU}(\bar{\mathbf{C}}), \qquad \tilde{\mathbf{G}} = \mathrm{PU}(\mathbf{G}),$$

the code constructs two additional heuristic priors in patch space:

$$\tilde{\mathbf{U}} = (1 - \tilde{\mathbf{C}}) \odot (1 - \tilde{\mathbf{D}}), \qquad \tilde{\mathbf{L}} = 1 - \tilde{\mathbf{C}}.$$

The full geometric input is then

$$\mathbf{X}_{geo} = \mathrm{Cat}\left[\tilde{\mathbf{D}}, \tilde{\mathbf{C}}, \tilde{\mathbf{G}}, \tilde{\mathbf{U}}, \tilde{\mathbf{L}}\right] \in \mathbb{R}^{M\times 5p^2\times h\times w}.$$

Because $p = 14$, this branch has $5p^2 = 980$ input channels. The geometric tensor is projected by a 1×1 convolution and ReLU:

$$\mathbf{P}_{geo} = \mathrm{ReLU}\big(\mathrm{Conv}_{geo}^{1\times1}(\mathbf{X}_{geo})\big) \in \mathbb{R}^{M\times C_e\times h\times w}.$$

The image and geometric branches are then fused by channel concatenation followed by a 3×3 convolution, batch normalization, and ReLU:

$$\mathbf{F}_{fuse} = \mathrm{ReLU}\left(\mathrm{BN}\left(\mathrm{Conv}^{3\times3}\left(\mathrm{Cat}[\mathbf{P}_{geo}, \mathbf{P}_{img}]\right)\right)\right) \in \mathbb{R}^{M\times C_e\times h\times w}.$$

Finally, the output block first reduces the channel dimension from 2048 to 1024 with a 3×3 convolution, batch normalization, and ReLU, then predicts $2p^2$ channels by a 1×1 convolution, and finally restores full resolution by `PixelShuffle`(14):

$$\mathbf{Z}_{patch} \in \mathbb{R}^{M\times 2p^2\times h\times w}, \qquad \mathbf{Z} = \mathrm{PixelShuffle}_p(\mathbf{Z}_{patch}) \in \mathbb{R}^{M\times 2\times H\times W}.$$

The final hard segmentation mask used at inference time is obtained by

$$\hat{\mathbf{M}}(u, v) = \arg\max_{c\in\{0,1\}} \mathbf{Z}_c(u, v).$$

Compared with earlier internal variants that relied only on summed DINO features or simple confidence gating, this final implementation injects explicit depth- and confidence-derived priors while still keeping the trainable head compact. During training, `DynaDA3.forward()` returns the logits $\mathbf{Z}$ directly; during inference, `DynaDA3.inference()` first executes the full DA3 pipeline and then attaches both $\mathbf{Z}$ and the hard mask $\hat{\mathbf{M}}$ to the prediction object.

3.1.3 Training

Training is restricted to the uncertainty segmentation head. The DA3 backbone, the Dual-DPT geometric head, and the optional camera branch remain frozen throughout optimization, and only the parameters of `model.uncertainty_head` are updated. In the training path, the input tensor must satisfy the legal DA3 size constraint $H = 14h$ and $W = 14w$; this is handled by data loading and cropping before the forward pass. Given an input batch

$\mathbf{I} \in \mathbb{R}^{B\times N\times 3\times H\times W}$, the frozen DA3 stage extracts four feature tensors together with depth and confidence maps, after which the wrapper flattens the batch and frame dimensions into $M = BN$. The uncertainty head therefore returns logits

$$\mathbf{Z} \in \mathbb{R}^{M\times 2\times H\times W},$$

that is, one two-class logit vector for every pixel of every sampled frame.

The supervision is formulated as pixel-wise binary classification with label 0 for certain or static pixels and label 1 for uncertain or dynamic pixels. Let $\mathbf{Y} \in \{0,1\}^{M\times H\times W}$ denote the ground-truth mask after the same flattening, and let $\mathbf{Z}_{m,c,u,v}$ be the pre-softmax logit of class c at pixel (u,v) in flattened frame m. The training objective is the standard per-pixel cross-entropy loss applied directly to logits:

$$\mathcal{L}_{unc} = -\frac{1}{MHW}\sum_{m=1}^{M}\sum_{u=1}^{H}\sum_{v=1}^{W}\log\frac{\exp\left(\mathbf{Z}_{m,\mathbf{Y}_{m,u,v},u,v}\right)}{\sum_{c=0}^{1}\exp(\mathbf{Z}_{m,c,u,v})}.$$

Equivalently, this can be understood as applying a softmax over the two channels of $\mathbf{Z}$ and penalizing the negative log-likelihood of the ground-truth class at every pixel. The code does not apply a sigmoid or threshold inside the model; instead, it returns raw logits during training and computes the hard mask only at inference time via an `argmax` over the class dimension.

The training data combines both generic dynamic-scene videos and endoscopic data. For the generic part, we use video sequences derived from the Wild-SLAM Mocap Dataset, the Wild-SLAM iPhone Dataset, and additional public sequences such as the Bonn dataset. For these data, binary uncertainty masks are generated with SAM3-assisted interactive segmentation and then manually corrected to obtain reliable labels. Beyond such general-scene data, we also include endoscopic and arthroscopic videos from datasets such as ArthroPhase, so that the uncertainty head is exposed to surgical appearance changes, instrument motion, fluid reflections, and other domain-specific disturbances. This mixed-data strategy improves the robustness of the uncertainty head without modifying the pretrained DA3 geometry model.

3.2 DynaDA3-SLAM

Building on the filtered geometry produced by DynaDA3, we further construct DynaDA3-SLAM as the real-time localization and navigation module of the overall system. In contrast to classical sparse-feature SLAM, the proposed method is formulated as a submap-level dense point-cloud SLAM pipeline. The system processes the input video in a fixed loop: incoming frames are read and optionally filtered by keyframe selection, a current submap with temporal overlap is formed, the learned front-end predicts geometry and prepares loop candidates, relative constraints are inserted into a global graph, the graph is optimized, and the refined global transforms are written back to the map. Consequently, the global optimization variables are submaps rather than individual frame poses, while the dense local geometry inside each submap is primarily provided by the learned front-end.

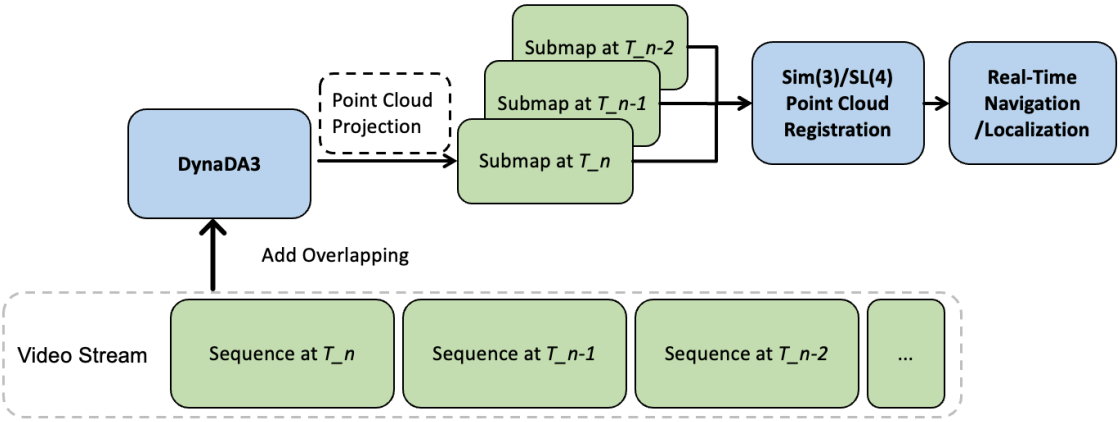

Figure 3.2: Overview of the proposed DynaDA3-SLAM architecture. The system builds on DynaDA3 outputs and integrates local submap construction, projective alignment, and global optimization for monocular endoscopic SLAM.

3.2.1 System Principle and Core Components

As illustrated in Figure 3.2, DynaDA3-SLAM is organized around a small number of tightly coupled modules. The class `Solver` acts as the main controller and coordinates batched inference, submap construction, inter-submap registration, loop-closure insertion, optimization, and visualization. The learned front-end is encapsulated in `DynaDA3`, which internally calls Depth Anything 3 to predict dense depth, confidence, camera extrinsics, and camera intrinsics, while additionally invoking the uncertainty head to suppress dynamic or geometrically unreliable regions. The module `UncertaintyDPT` performs this binary segmentation

by fusing depth, confidence, depth gradients, and backbone features. For frame selection, `FrameTracker` evaluates motion magnitude from Shi–Tomasi corners tracked by pyramidal Lucas–Kanade optical flow. Loop detection is handled by `ImageRetrieval`, which extracts global descriptors with SALAD/DINOv2. The class `Submap` stores local dense geometry, local camera poses, colors, confidences, masks, and the submap-to-world transform, whereas `GraphMap` manages the collection of all submaps and writes optimized transforms back to the map. Finally, `PoseGraph` provides the back-end graph optimization.

3.2.2 Image Scheduling and Keyframe Selection

The video stream is first ordered by frame index and then processed in windows of size `submap_size + overlapping_window_size`. After a submap has been processed, only the tail overlap is retained as the starting segment of the next window. This overlap mechanism guarantees geometric continuity between neighboring submaps while keeping the memory footprint bounded. Before a frame is accepted into the current processing batch, the system may optionally apply keyframe selection. Specifically, `FrameTracker` detects Shi–Tomasi corners and tracks them with pyramidal Lucas–Kanade optical flow. The average pixel displacement of the surviving tracks is used as a disparity measure, and the frame is retained only if the motion exceeds a predefined threshold. In this way, the system avoids feeding highly redundant frames into the front-end while still preserving sufficient parallax for dense reconstruction and registration.

3.2.3 Learned Front-End Inference and Dense Submap Reconstruction

For each processing window, `Solver` creates a new `Submap` instance and records the corresponding frames and frame indices. Before learned inference is executed, the current frames are passed through `ImageRetrieval` to compute SALAD/DINOv2 global descriptors and to search for loop-closure candidates in the historical map. This search is performed against previously stored submaps while skipping the immediately preceding submap so that ordinary overlap is not mistaken for a loop. If candidate loop frames are found, they are appended to the current inference batch as auxiliary historical frames rather than being reconstructed as new submaps. The implementation therefore distinguishes between newly processed frames and loop-support frames inside the same temporary batch.

The batched images are then forwarded to `DynaDA3.inference()`, which produces two groups of outputs. First, the monocular geometric branch predicts dense depth, per-pixel confidence, and camera extrinsics and intrinsics. Second, the uncertainty branch estimates a binary mask of dynamic or uncertain regions. Internally, `UncertaintyDPT` performs robust percentile-based normalization of depth and confidence, computes depth gradients, converts local patch structure to channel features via `pixel_unshuffle`, and fuses these signals with backbone features to predict the mask. After inference, the depth and camera parameters are rescaled to the original image resolution. Dense 3D points are then reconstructed by back-projecting each pixel through the pinhole camera model and transforming the resulting point cloud from camera coordinates to the local submap frame. The output is therefore a dense per-pixel point cloud representation rather than a sparse landmark set. Dynamic and uncertain pixels identified by the mask are excluded from later registration steps so that unstable geometry does not contaminate the submap constraints.

3.2.4 Inter-Submap Registration and Loop-Closure Constraints

Once the current predictions are available, `Solver.add_points()` inserts the submap into the global map and constructs graph constraints. The first submap is anchored by an identity prior. For every later submap, an adjacent constraint is estimated from the overlap between the first frame of the current submap and the retained overlap frames of the previous submap. Because the monocular reconstruction is not guaranteed to be metrically rigid, the default implementation estimates a 3D projective homography $H \in SL(4)$ rather than a pure $SE(3)$ transform. The solver uses a RANSAC-based projective estimator: five 3D–3D correspondences are sampled, a linear homography system is solved in closed form, and the hypothesis is scored by Euclidean 3D alignment error over all available points. The best transformation is added as a between-factor between the two submap nodes.

Loop-closure constraints are created in a similar way, but now the matched geometry comes from two different sources: the original historical frame already stored in an old submap, and the same historical frame re-inferred inside the current batch as a loop-support frame. This design ensures that loop edges are also estimated at the submap level. The resulting loop transformation is added as another between-factor in the graph. As a consequence, DynaDA3-SLAM does not build a frame-level pose graph; instead, both sequential and loop constraints connect submap nodes whose internal dense geometry is provided by the learned front-end.

3.2.5 Global Graph Optimization and Map Update

After the adjacent and loop constraints have been inserted, the global graph is optimized immediately. The back-end is implemented with a GTSAM non-linear factor graph and solved using the Levenberg–Marquardt optimizer. In the default configuration, the graph operates on custom $SL(4)$ nodes with corresponding prior and between factors, so the system performs optimization directly in projective transformation space rather than on a classical $SE(3)$ pose graph. A $Sim(3)$ branch remains available in the implementation, but the standard configuration used in this work follows the projective formulation described above.

After convergence, the optimized submap-to-world transformations are written back to the stored submaps by the global map manager. The global map is then recovered by transforming each local dense point cloud into the world frame with its refined submap transform. Likewise, world-coordinate camera poses are obtained by combining the local camera estimates inside each submap with the optimized submap-to-world transformation. Importantly, the back-end corrects only the relative placement of submaps in the global scene. The internal dense geometry of each submap and the relative structure between its frames remain primarily determined by the learned front-end, so the system does not perform a classical global bundle adjustment over all frames and pixels.

4 Results and Discussion

This chapter presents the benchmark categories that are directly relevant to the proposed system, namely pose estimation, depth reconstruction, 3D scene quality, SLAM, and dynamic obstacle prediction. Although the cited methods are not always evaluated on exactly the same datasets, they report the same types of experiments targeted by DynaDA3 and DynaDA3-SLAM. Therefore, their published results are organized here as task-consistent literature baselines, while dedicated rows are reserved for the final results of the proposed method.

4.1 Pose Estimation Results

Accurate pose recovery is a prerequisite for both geometric reconstruction and SLAM. For this reason, short-window pose estimation is first compared with representative monocular endoscopic methods. Table 4.1 summarizes the pose benchmark reported on SCARED by Endo3R [30], where the task is to recover stable camera motion over 5-frame snippets.

Table 4.1: Comparison of pose estimation results on SCARED (5-frame evaluation), reproduced from Endo3R [30].

Method	**ATE** $\downarrow$	$RPE_r \downarrow$	$RPE_t \downarrow$
Endo-SfM	0.157	0.252	0.259
AF-SfM	0.125	0.235	0.241
EndoDAC	0.124	0.223	0.233
Robust	0.131	0.241	0.245
Endo3R	0.112	0.201	0.228
DynaDA3 (Ours)		*TBD*	

Among the methods listed in Table 4.1, Endo3R reports the lowest ATE and the best overall short-window pose consistency. This result indicates that feed-forward geometry models can already provide competitive motion recovery before being integrated into a full SLAM pipeline.

In parallel, Endo3DAC [33] reports a mean ATE of 0.0583 on SCARED under a similar short-sequence pose evaluation, which further supports the use of recent geometry foundation models as strong pose baselines for the proposed method.

4.2 Depth Reconstruction Results

Depth prediction is treated as an independent benchmark because it measures the frame-wise geometric quality produced by the model before any global fusion or mapping stage. For the proposed system, this experiment answers how accurately DynaDA3 can recover local scene geometry from endoscopic video.

Table 4.2 summarizes representative depth-oriented reconstruction results from Endo3R [30]. To keep the comparison focused, only a compact set of strong baselines is retained, including EndoDAC, Endo DM, Monst3R, and Endo3R. These methods cover endoscopy-specific adaptation, more general monocular reconstruction, and recent feed-forward geometric modeling.

Table 4.2: Comparison of depth-oriented reconstruction results on SCARED and Hamlyn, condensed from Endo3R [30].

Dataset	**Method**	**Abs Rel** $\downarrow$	**RMSE** $\downarrow$	$\delta < 1.25 \uparrow$	**FPS** $\uparrow$
SCARED	EndoDAC	0.242	2.014	0.584	31.79
SCARED	Endo DM	0.203	2.063	0.612	14.58
SCARED	Monst3R	0.198	1.965	0.626	18.68
SCARED	Endo3R	0.124	1.209	0.839	19.17
SCARED	DynaDA3 (Ours)		*TBD*		
Hamlyn	EndoDAC	0.275	15.669	0.519	31.79
Hamlyn	Endo DM	0.216	14.799	0.619	14.58
Hamlyn	Monst3R	0.198	15.221	0.645	18.68
Hamlyn	Endo3R	0.170	11.569	0.707	19.17
Hamlyn	DynaDA3 (Ours)		*TBD*		

On both SCARED and Hamlyn, Endo3R reports the best overall balance between reconstruction accuracy and online efficiency among the listed methods. In particular, its substantial improvement in Abs Rel and RMSE suggests that explicitly learning temporally aligned geometry is beneficial for challenging endoscopic video. These results define the most relevant

reconstruction reference range for DynaDA3, whose final quantitative results will be inserted under the same experiment type.

4.3 3D Scene Quality Results

Per-frame depth metrics do not completely capture the global quality of the reconstructed scene. Therefore, Table 4.3 additionally reports explicit 3D reconstruction metrics taken from Endo3DAC [33]. This comparison evaluates the reconstructed point cloud or scene geometry directly through accuracy, completeness, chamfer distance, precision, recall, and F_1 score.

Table 4.3: Comparison of 3D scene reconstruction results on SCARED, reproduced from Endo3DAC [33].

Method	**Acc** $\downarrow$	**Comp** $\downarrow$	**Cham** $\downarrow$	**Prec** $\uparrow$	**Rec** $\uparrow$	F_1 $\uparrow$
MonoGS	3.56	2.54	2.97	70.0	81.4	75.2
FrozenRecon	4.91	2.59	3.25	74.0	90.7	77.5
Endo3DAC	3.08	1.86	2.66	77.7	88.3	82.6
DynaDA3 (Ours)			*TBD*			

Compared with MonoGS and FrozenRecon, Endo3DAC achieves the best overall geometric consistency in this benchmark, especially in completeness, chamfer distance, and F_1 score. This indicates that stronger depth and pose coupling can translate into more usable reconstructed geometry. For the proposed method, this table will be the natural location to report whether dynamic filtering further improves the final 3D scene quality.

4.4 SLAM Results

The SLAM comparison is organized into sparse and dense settings. Sparse SLAM results are particularly useful for judging trajectory robustness on joint sequences, whereas dense SLAM results reveal the trade-off between localization, mapping quality, and online visualization. Together, these two perspectives cover the main functional requirements of DynaDA3-SLAM.

4.4.1 Sparse SLAM Results on Joint Sequences

Table 4.4 reproduces the joint-domain SLAM results reported by OneSLAM [35]. This comparison is especially relevant for the present thesis because the dataset category is closer to arthroscopy-like joint scenes than most colonoscopy-only benchmarks.

Table 4.4: Comparison of sparse SLAM results on joint sequences, reproduced from OneSLAM [35].

Method	**Trans. ATE** ↓	**Rot. ATE** ↓	**Trans. RPE** ↓	**Rot. RPE** ↓	**Tracked Cameras** ↑
OneSLAM	0.93	16.07	0.49	1.05	100%
EndoSLAM	2.50	73.64	1.09	14.38	100%
ORB-SLAM3	0.80	63.39	0.91	1.67	20%
DynaDA3-SLAM (Ours)			*TBD*		

The results in Table 4.4 show that OneSLAM maintains full trajectory coverage while keeping low translational drift on joint sequences. ORB-SLAM3 achieves a slightly lower translational ATE, but only tracks 20% of the cameras, which strongly limits its practical usefulness. This comparison highlights that robust coverage and trajectory continuity are as important as pure error minimization in endoscopic SLAM, and these aspects should therefore be reported explicitly for DynaDA3-SLAM.

4.4.2 Dense SLAM Results

For dense SLAM, EndoGSLAM [36] provides a representative benchmark on C3VD. Table 4.5 compares rendering quality, depth reconstruction quality, and localization accuracy in a unified dense online setting.

Table 4.5: Comparison of dense SLAM results on C3VD, reproduced from EndoGSLAM [36].

Method	**PSNR** ↑	**SSIM** ↑	**LPIPS** ↓	**RMSE (mm)** ↓	**ATE (mm)** ↓
ORB-SLAM3	17.89	0.64	0.35	7.72	0.32
NICESLAM	22.07	0.73	0.33	1.88	0.48
Endo-Depth	18.13	0.64	0.33	5.10	1.25
EndoGSLAM-H	22.16	0.77	0.22	2.17	0.34
EndoGSLAM-R	18.37	0.67	0.30	4.33	1.23
DynaDA3-SLAM (Ours)			*TBD*		

EndoGSLAM-H achieves the strongest overall rendering and perceptual quality, whereas ORB-SLAM3 remains competitive only in camera localization error. This result reflects a common

pattern in endoscopic SLAM: sparse systems may still localize well, but dense scene quality and online visualization require a different design emphasis. EndoGSLAM further reports tracking/reconstruction times of 151.4/268.0 ms per frame for the high-quality version and 62.4/65.1 ms per frame for the real-time version, while maintaining online rendering above 100 fps [36]. These numbers provide an important runtime reference for the future evaluation of DynaDA3-SLAM.

4.5 Dynamic Obstacle Prediction Results

Besides geometry and tracking, the proposed system explicitly addresses dynamic occlusions. This task is not typically evaluated in the same standardized way as reconstruction or SLAM in the cited literature, but it is essential for the intended surgical setting of the present work. Therefore, the obstacle prediction head is treated as an independent experimental component, and its accuracy will be measured by comparing the predicted binary masks against SAM3-based masks that are manually refined on representative frames.

Table 4.6: Planned quantitative results for dynamic obstacle prediction.

Method	**IoU** ↑	**Dice** ↑	**Precision** ↑	**Recall** ↑
DynaDA3 Motion Mask (Ours)			*TBD*	

In addition to the quantitative comparison, qualitative examples should later be included for typical dynamic cases such as tool insertion, sudden occlusion, partial tissue motion, and highlight-corrupted frames. Such examples are important because they reveal whether the mask head suppresses only true dynamic distractors or also removes geometrically useful static tissue regions.

4.6 Discussion

Taken together, the literature results show that recent endoscopic geometry models have already achieved strong performance in short-window pose estimation, depth prediction, and 3D scene reconstruction quality, while endoscopic SLAM systems still expose a clear trade-off between trajectory robustness, dense scene quality, and real-time efficiency. The proposed DynaDA3 and DynaDA3-SLAM are designed to connect these two lines of development by

combining feed-forward geometric prediction with explicit dynamic filtering. Once the experiments of the proposed system are completed, the reserved entries in this chapter will allow a direct comparison under the same experiment types discussed above.

5 Conclusion and Outlook

Bibliography

[1] S.-W. Yoo. "Neural recognition and 3D-reconstruction pipeline for automated replacement of knee cartilage tissue". MA thesis. Aachen, Germany: Chair of Chemical Process Engineering, RWTH Aachen University, May 2024 (cit. on pp. 1, 3, 5–7).

[2] J. M. Oliveira and R. L. Reis, eds. *Regenerative Strategies for the Treatment of Knee Joint Disabilities*. Vol. 21. Studies in Mechanobiology, Tissue Engineering and Biomaterials. Cham: Springer, 2017. DOI: 10.1007/978-3-319-44785-8 (cit. on pp. 3, 5).

[3] A. J. Sophia Fox, A. Bedi, and S. A. Rodeo. "The basic science of articular cartilage: structure, composition, and function". *Sports Health* 1.6 (2009) 461–468. DOI: 10.1177/1941738109350438 (cit. on pp. 4, 5).

[4] U. Kjellén and U. Lindahl. "Proteoglycans: structures and interactions". *Annual Review of Biochemistry* 60.1 (1991) 443–475. DOI: 10.1146/annurev.bi.60.070191.002303 (cit. on p. 4).

[5] S. Akizuki, V. C. Mow, F. Müller, J. C. Pita, D. S. Howell, and D. H. Manicourt. "Tensile properties of human knee joint cartilage: I. Influence of ionic conditions, weight bearing, and fibrillation on the tensile modulus". *Journal of Orthopaedic Research* 4.4 (1986) 379–392. DOI: 10.1002/jor.1100040401 (cit. on p. 4).

[6] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny. "VGGT: Visual Geometry Grounded Transformer". In*Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025, 5294–5306. DOI: 10.1109/CVPR52734.2025.00499 (cit. on pp. 9, 10, 13, 33).

[7] P. J. Besl and N. D. McKay. "A Method for Registration of 3-D Shapes". *IEEE Transactions on Pattern Analysis and Machine Intelligence* 14.2 (1992) 239–256. (Cit. on p. 11).

[8] R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. 2nd ed. Cambridge University Press, 2004 (cit. on p. 11).

[9] S. Umeyama. "Least-Squares Estimation of Transformation Parameters Between Two Point Patterns". *IEEE Transactions on Pattern Analysis and Machine Intelligence* 13.4 (1991) 376–380. (Cit. on p. 13).

[10] D. Maggio, H. Lim, and L. Carlone. "VGGT-SLAM: Dense RGB SLAM Optimized on the SL(4) Manifold". *Advances in Neural Information Processing Systems* 39 (2025) (cit. on pp. 13–15, 17, 25, 26).

[11] M. A. Fischler and R. C. Bolles. "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography". *Communications of the ACM* 24.6 (1981) 381–395. (Cit. on p. 15).

[12] D. Nistér. "An efficient solution to the five-point relative pose problem". *IEEE Transactions on Pattern Analysis and Machine Intelligence* 26.6 (2004) 756–770. (Cit. on pp. 15, 25).

[13] J. Matas and O. Chum. "Randomized RANSAC with T d, d test". *Image and vision computing* 22.10 (2004) 837–842. (Cit. on pp. 15, 25).

[14] B. Triggs, P. F. McLauchlan, R. I. Hartley, and A. W. Fitzgibbon. "Bundle Adjustment – A Modern Synthesis". In*Vision Algorithms: Theory and Practice*. Springer, 2000. 298–372. (Cit. on p. 16).

[15] S. Agarwal, N. Snavely, S. M. Seitz, and R. Szeliski. "Bundle Adjustment in the Large". In*European Conference on Computer Vision (ECCV)*. 2010, 29–42 (cit. on p. 16).

[16] R. Kummerle, G. Grisetti, H. Strasdat, K. Konolige, and W. Burgard. "g2o: A General Framework for Graph Optimization". In*Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2011, 3607–3613 (cit. on p. 17).

[17] F. Dellaert. *Factor Graphs and GTSAM: A Hands-on Introduction*. Tech. rep. GT-RIM-CP&R-2012-002. Atlanta, GA, USA: Georgia Institute of Technology, Center for Robotics and Intelligent Machines, 2012 (cit. on p. 17).

[18] H. Lin, S. Chen, J. H. Liew, D. Y. Chen, Z. Li, G. Shi, J. Feng, and B. Kang. "Depth Anything 3: Recovering the Visual Space from Any Views". *arXiv preprint arXiv:2511.10647* (2025) DOI: 10.48550/arXiv.2511.10647 (cit. on pp. 18, 24, 25, 32, 36, 37).

[19] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard. "Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age". *IEEE Transactions on Robotics* 32.6 (2016) 1309–1332. (Cit. on pp. 19, 20).

[20] G. Klein and D. Murray. "Parallel Tracking and Mapping for Small AR Workspaces". In*Proceedings of the 6th IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR)*. 2007, 225–234 (cit. on p. 19).

[21] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos. "ORB-SLAM: a Versatile and Accurate Monocular SLAM System". *IEEE Transactions on Robotics* 31.5 (2015) 1147–1163. (Cit. on pp. 19, 20).

[22] R. Mur-Artal and J. D. Tardos. "ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras". *IEEE Transactions on Robotics* 33.5 (2017) 1255–1262. (Cit. on pp. 19, 20).

[23] J. Engel, T. Schops, and D. Cremers. "LSD-SLAM: Large-Scale Direct Monocular SLAM". In*European Conference on Computer Vision (ECCV)*. 2014, 834–849 (cit. on p. 19).

[24] J. Engel, V. Koltun, and D. Cremers. "Direct Sparse Odometry". *IEEE Transactions on Pattern Analysis and Machine Intelligence* 40.3 (2018) 611–625. (Cit. on p. 19).

[25] A. Dosovitskiy et al. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". In*International Conference on Learning Representations (ICLR)*. 2021 (cit. on p. 22).

[26] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin. "Emerging Properties in Self-Supervised Vision Transformers". In*Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021, 9650–9660. DOI: 10.1109/ICCV48922.2021.00951 (cit. on pp. 22, 23).

[27] R. Ranftl, A. Bochkovskiy, and V. Koltun. "Vision Transformers for Dense Prediction". In*Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021, 12179–12188. DOI: 10.1109/ICCV48922.2021.01196 (cit. on p. 23).

[28] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud. "DUST3R: Geometric 3D vision made easy". In*IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024, 20697–20709 (cit. on p. 25).

[29] E. Weiszfeld. "Sur le point pour lequel la somme des distances de n points donnés est minimum". *Tohoku Mathematical Journal, First Series* 43 (1937) 355–386. (Cit. on p. 25).

[30] J. Guo, W. Dong, T. Huang, H. Ding, Z. Wang, H. Kuang, Q. Dou, and Y.-H. Liu. "Endo3R: Unified Online Reconstruction from Dynamic Monocular Endoscopic Video". In*Medical Image Computing and Computer Assisted Intervention – MICCAI 2025*. Vol. 15968. Lecture Notes in Computer Science. Springer Nature Switzerland, 2025, 170–180. DOI: 10.1007/978-3-032-05114-1_17 (cit. on pp. 32, 46, 47).

[31] B. Cui, M. Islam, L. Bai, A. Wang, and H. Ren. "EndoDAC: Efficient Adapting Foundation Model for Self-Supervised Depth Estimation from Any Endoscopic Camera". In*Medical Image Computing and Computer Assisted Intervention – MICCAI 2024*. Vol. 15006. Lecture Notes in Computer Science. Springer Nature Switzerland, 2024, 208–218. DOI: 10.1007/978-3-031-72089-5_20 (cit. on p. 32).

[32] Y. Li, D. Chang, D. Luo, J. Huang, L. Dong, D. Wang, L. Mei, and C. Lei. "SfMDiffusion: self-supervised monocular depth estimation in endoscopy based on diffusion models". *International Journal of Computer Assisted Radiology and Surgery* 20.5 (2025) 971–979. DOI: 10.1007/s11548-025-03333-0 (cit. on p. 32).

[33] B. Cui et al. "Learning to Efficiently Adapt Foundation Models for Self-Supervised Endoscopic 3D Scene Reconstruction from Any Cameras". *arXiv preprint arXiv:2503.15917* (2025) DOI: 10.48550/arXiv.2503.15917 (cit. on pp. 32, 47, 48).

[34] J. Huo, C. Zhou, B. Yuan, Q. Yang, and L. Wang. "Real-Time Dense Reconstruction with Binocular Endoscopy Based on StereoNet and ORB-SLAM". *Sensors* 23.4 (2023) 2074. DOI: 10.3390/s23042074 (cit. on p. 33).

[35] T. Teufel, H. Shu, R. D. Soberanis-Mukul, J. E. Mangulabnan, M. Sahu, S. S. Vedula, M. Ishii, G. Hager, R. H. Taylor, and M. Unberath. "OneSLAM to map them all: a generalized approach to SLAM for monocular endoscopic imaging based on tracking any point". *International Journal of Computer Assisted Radiology and Surgery* 19.7 (2024) 1259–1266. DOI: 10.1007/s11548-024-03171-6 (cit. on pp. 33, 49).

[36] K. Wang, C. Yang, Y. Wang, S. Li, Y. Wang, Q. Dou, X. Yang, and W. Shen. "EndoGSLAM: Real-Time Dense Reconstruction and Tracking in Endoscopic Surgeries using Gaussian Splatting". In*Medical Image Computing and Computer Assisted Intervention – MICCAI 2024*. Vol. 15006. Lecture Notes in Computer Science. Springer Nature Switzerland, 2024, 219–229. DOI: 10.1007/978-3-031-72089-5_21 (cit. on pp. 33, 49, 50).

[37] M. Turan, Y. Almalioglu, H. Gilbert, H. Araujo, E. Konukoglu, and M. Sitti. "Magnetic-Visual Sensor Fusion based Medical SLAM for Endoscopic Capsule Robot". *arXiv preprint arXiv:1705.06196* (2017) DOI: 10.48550/arXiv.1705.06196 (cit. on p. 33).

[38] Z. Li, M. Cui, D. Hu, J. Gong, J. Weng, Z. Zhang, L. Tian, M. Li, and K. Huang. "A Two-Stage Method for Specular Highlight Detection and Removal in Medical Images". In*Medical Image Computing and Computer Assisted Intervention – MICCAI 2025*. Vol. 15969. Lecture Notes in Computer Science. Springer Nature Switzerland, 2025, 23–33. DOI: 10.1007/978-3-032-05127-1_3 (cit. on pp. 35, 36).

List of Figures

2.1 Schematic overview of the knee joint and the anatomical location of articular cartilage. 4
2.2 Illustration of a focal cartilage lesion in the knee joint. 6
2.3 Comparison between a conventional multi-step cartilage repair workflow and the TriggerINK single-surgery robotic bioprinting concept. 7
2.4 Structure-from-Motion pipeline 10
2.5 Standard visual SLAM pipeline 21
2.6 DUSt3R framework overview 26
2.7 VGGT framework overview 27
2.8 DA3 framework overview 28
2.9 DA3 Dual-DPT Head 30

3.1 DynaDA3 architecture 36
3.2 DynaDA3-SLAM architecture 42

List of Tables

2.1 Overview of State-of-the-Art Methods for Endoscopic Dense Reconstruction and Endoscopic SLAM 31

4.1 Pose estimation results on SCARED 46
4.2 Depth reconstruction results 47
4.3 3D reconstruction results on SCARED 48
4.4 Joint-sequence SLAM results 49
4.5 Dense SLAM results on C3VD 49
4.6 Dynamic obstacle prediction results 50

A Appendix